DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

COURSE: DATABASE MANAGEMENT SYSTEMS (DBMS) | ACADEMIC YEAR: 2025–2026

SMARTRETAIL
AI-POWERED INVENTORY AND SALES MANAGEMENT SYSTEM

DATABASE MANAGEMENT SYSTEMS
DIGITAL ASSIGNMENT 2 (DA2) COMPREHENSIVE PROJECT REPORT

STUDENT NAME	REGISTRATION NUMBER
SRI PRIYAN D	25BCE1074
NAIDU RESHMANTH SAI	25BCE1112

Target RDBMS Engine:	Oracle Database 11g Express Edition Release 11.2.0.2.0 (64-bit)
Target Schema User:	SYSTEM (Verified Local Project Account)
Implementation Scope:	19 Approved Relations (Normalized Relational Model)
Primary Source of Truth:	SmartRetail_DA2_Antigravity_Handoff.docx
Relational Normalization:	Boyce-Codd Normal Form (BCNF) Verified Across All Relations
Empirical Test Results:	13 / 13 Automated Test Scenarios Verified PASSED (0 Failures)

Under the Guidance of:
[Faculty Name], Department of Computer Science & Engineering

Table of Contents

This technical document presents the complete system architecture, relational database design, rigorous normalization proof (UNF to BCNF), and empirical execution evidence for the SmartRetail system implemented on Oracle Database 11g Express Edition Release 11.2.0.2.0.

Report Section	Contents & Scope Overview
1. Introduction	System background, retail domain scope, architectural context
2. Problem Statement	Operational challenges, data consistency, multi-tier inventory hazards
3. Project Objectives	11 Technical milestones for design, normalization, implementation, and testing
4. System Scope & Functional Requirements	Subsystem boundaries, core functional requirements, technical constraints
5. Conceptual & Logical Database Design	Entity classification, cardinalities, ER & EER diagram models
6. Relational Normalization Analysis	Comprehensive decomposition from UNF, 1NF, 2NF, 3NF through BCNF
7. Formal Functional Dependencies Specification	Rigorous functional dependencies for all 19 approved relations
8. Complete Relational Schema Specification	Data dictionary tables for Procurement, Catalog, Inventory, and POS subsystems
9. SQL Implementation & Empirical Verification	DDL, Sequences, Seed Data, Joins, Aggregation, Advanced SQL, Views, Indexes
10. PL/SQL Stored Program Units Implementation	Procedures, Functions, Cursors, Exceptions, Triggers, and Transactions
11. Business Workflows & Operational Sequences	Inbound Procurement, Internal Shelf Replenishment, and POS Checkout flows
12. Automated Testing & Verification Suite	13-Scenario test matrix, regression validation, empirical 13/13 PASS summary
13. Execution Evidence Catalog	Catalog of all 17 embedded figures and SQL*Plus console evidence
14. Conclusion	Summary of academic milestones, normalization rigor, and database robustness
15. References & Project Sources	Authoritative course documentation, source handoff brief, and Oracle specifications

1. Introduction

SmartRetail is a normalized relational database management system developed to model, coordinate, and execute core business operations within modern supermarket retail environments. Modern retail establishments handle high volumes of concurrent transactions across diverse operational areas, necessitating a unified and logically sound data architecture. In traditional legacy setups, data fragmentation between warehouse storage, procurement logistics, and front-of-store retail counters frequently results in data inconsistencies, stockouts, inaccurate inventory balances, and compromised transactional integrity.

The SmartRetail database architecture addresses these challenges by consolidating the full operational lifecycle into a cohesive relational structure. Specifically, the system manages twelve core domains of retail operations:

- **Suppliers and Logistics Channels:** Commercial vendor identification, tax compliance registrations (GST), contact channels, and vendor reliability ratings.
- **Merchandise Catalog and Categorization:** Standardized product definitions, unique Barcode/SKU identification, units of measure, active commercial status, and recursive merchandise classifications.
- **Procurement and Receiving:** Purchase order authoring, contracted line-item quantities, inbound shipment dispatch tracking, and partial/full warehouse receiving reconciliation.
- **Multi-Tier Physical Storage:** Clear physical and logical segregation between bulk storage warehouse facilities and customer-accessible display shelf locations.
- **Real-Time Stock Ledgers:** Location-specific balances, reorder threshold values, and predetermined replenishment quantities.
- **Stock Movements and Audit Tracking:** An immutable transaction ledger recording every addition, deduction, transfer, and manual adjustment.
- **Automated Replenishment Workflows:** Trigger-driven automated event generation when stock falls to or below threshold levels, initiating internal shelf restocking or supplier procurement.
- **Customer Account Profiles:** Customer identification, registration metadata, membership tiers, and loyalty point balances.
- **Point of Sale (POS) Billing:** Transaction invoice generation, checkout cashier attribution, itemized sales lines, and support for guest shoppers.
- **Multi-Tender Payment Settlement:** Split payment processing allowing invoices to be settled across multiple tenders (Cash, Card, UPI, Wallet, Points).
- **Store Personnel Directory:** Employee records with strict role-based operational segregation across Management, Cashier, Inventory Clerk, and Administration functions.
- **System Conformance and Integrity:** Declarative primary, foreign, unique, and domain check constraints reinforced by transactional database triggers.

The implementation presented in this report is fully deployed and validated on Oracle Database 11g Express Edition Release 11.2.0.2.0. The design strictly preserves the 19 approved relational entities from Digital Assignment 1 (DA1), achieving rigorous compliance with Boyce-Codd Normal Form (BCNF) principles.

2. Problem Statement

Supermarket operations operate in high-throughput environments where inventory velocity is fast and transaction accuracy is vital. Managing these operations with disparate or poorly normalized database models gives rise to significant database and operational problems:

- **Multi-Location Inventory Desynchronization:** In a retail environment, products exist in two fundamentally different physical zones: backroom/central bulk warehouses and customer-accessible retail display shelves. Systems that track

a single global quantity field fail to identify when front-of-store display shelves are empty while bulk warehouses are full, resulting in lost sales and unfulfilled customer demand.

- **Procurement and Receiving Discrepancies:** Vendors frequently fulfill purchase orders through partial deliveries across multiple shipments. Without itemized shipment tracking that reconciles shipped quantities against received quantities at the line-item level, inventory records become inaccurate, leading to incorrect supplier payments or missing stock.
- **Lack of Immutable Audit Trails:** Direct in-place updates to stock quantity columns destroy historical visibility. When discrepancies arise between physical stock and digital counts, administrators cannot audit whether deviations were caused by point-of-sale checkouts, internal shrinkage, warehouse transfers, or receiving mistakes.
- **Manual Replenishment Latency:** Relying on physical shelf inspections or end-of-day batch reports to identify low-stock conditions introduces unacceptable latency. Depleted shelves remain unstocked for hours, causing immediate revenue loss.
- **POS Invoicing and Payment Coupling:** Rigid 1:1 relationships between checkout invoices and single payment methods prevent split-tender settlement, where a customer pays partially in cash or card and settles the remainder via digital payment platforms (UPI or store points).
- **Concurrency and Data Integrity Hazards:** Simultaneous operations—such as POS cashiers dispensing items from shelves while warehouse personnel execute internal restocking transfers—risk race conditions, negative inventory states, and lost updates without declarative domain constraints and ACID-compliant transactional controls.

> **Core DBMS Problem Formulation:** To formulate and implement a fully normalized (BCNF), multi-tier relational schema and procedural database framework in Oracle Database 11g XE that guarantees physical inventory visibility across warehouses and shelves, maintains an immutable transaction movement ledger, supports multi-tender split payments, and automates threshold-driven replenishment without manual intervention or data corruption.

3. Project Objectives

The primary objective of this project is to develop, normalize, implement, and empirically validate an enterprise-grade relational database for SmartRetail. The concrete technical milestones are:

#	Milestone	Detailed Technical Scope & Deliverable
1	Schema Design & Normalization	Formulate a normalized relational schema strictly adhering to Boyce-Codd Normal Form (BCNF) across all 19 approved relations, eliminating update, insertion, and deletion anomalies.
2	Vendor & Procurement Modeling	Model supplier registries, multi-valued contact channels, purchase orders, and itemized purchase order lines with contracted procurement unit pricing.
3	Inbound Logistics Reconciliation	Implement shipment tracking and shipment line reconciliation to record carrier movements, partial fulfillment, and warehouse receiving confirmation.
4	Multi-Tier Location Segregation	Establish physical and logical distinction between bulk WAREHOUSE storage and front-of-store display SHELF locations within a self-referential hierarchy.
5	Immutable Inventory Auditing	Maintain an append-only audit ledger in InventoryTransaction capturing every stock addition, deduction, transfer, and adjustment with foreign-key referential binding.
6	Point-of-Sale & Split Tender	Design point-of-sale receipting supporting cashier attribution, itemized sales lines, optional customer tracking, and 1:N multi-tender split payment settlement.
7	Declarative Integrity Enforcement	Implement primary, foreign, unique, not null, and domain check constraints enforcing valid status domains, positive quantities, and active merchandise states.

#	Milestone	Detailed Technical Scope & Deliverable
8	PL/SQL Procedural Operations	Develop stored procedures (PROCESS_SALE, RECEIVE_SHIPMENT, TRANSFER_STOCK, CREATE_PURCHASE_ORDER) encapsulating atomic, multi-table transactions with custom exception handling.
9	Analytical Functions & Cursors	Implement PL/SQL stored functions for valuation/totals and demonstrate explicit, cursor FOR loop, and parameterized cursor reporting paradigms.
10	Trigger-Driven Event Automation	Implement database triggers ensuring synchronous order totals synchronization and reactive event logging for depleted shelves (RestockEvent) and warehouses (ReorderTrigger).
11	Empirical Test Verification	Construct and execute a 13-scenario automated regression test suite validating referential integrity, check constraints, procedural execution, and rollback consistency against Oracle 11g XE.

4. System Scope & Functional Requirements

The operational scope of the SmartRetail database encompasses four interconnected functional subsystems, establishing rigorous boundary conditions and system expectations:

4.1 Functional Subsystems

- **1. Procurement & Supplier Relations Subsystem:** Covers commercial vendor onboarding, multi-valued telephone communication channels, purchase order placement, procurement cost tracking, and inbound shipment receiving reconciliation.
- **2. Catalog & Merchandising Subsystem:** Covers product categorization across hierarchical parent-child categories, master SKU/barcode catalog management, unit of measure standards, and active retail pricing.
- **3. Multi-Tier Inventory & Replenishment Subsystem:** Tracks live stock balances across bulk warehouses and front-of-store shelves, records immutable stock movement transactions, and generates automated restock and reorder events based on threshold rules.
- **4. Point-of-Sale (POS) Checkout & Billing Subsystem:** Handles receipt generation, itemized sales line recording, cashier attribution, customer loyalty profile integration, and multi-tender split payment settlement across Cash, Credit Card, Debit Card, UPI, and Wallet.

4.2 Technical Environment & Operational Constraints

- **RDBMS Engine:** Oracle Database 11g Express Edition Release 11.2.0.2.0 (64-bit production release on Microsoft Windows).
- **Surrogate Key Architecture:** Surrogate primary keys are incremented via Oracle SEQUENCE objects and BEFORE INSERT triggers with WHEN (NEW.<col> IS NULL) clauses to ensure non-invasive auto-generation.
- **Zero Double-Deduction Principle:** Inventory stock deductions are strictly managed through procedural business logic (PROCESS_SALE, TRANSFER_STOCK) rather than cascading triggers to prevent double-decrement anomalies.
- **Security & Credential Management:** Zero plain-text credentials or database passwords are hard-coded into scripts, documentation, or reports. All execution connects via authenticated project accounts.

5. Conceptual & Logical Database Design

Conceptual database design establishes the high-level semantic model of the enterprise, identifying core entities, attributes, primary identity mechanisms, and structural relationships. In SmartRetail, real-world retail transactions require capturing both physical product handling (storage and replenishment) and commercial commitments (procurement contracts and POS checkout invoices).

5.1 Entity Identification and Relationship Semantics

The system models core operational entities categorized into four structural domains:

- **Independent Master Entities:** Supplier, ProductCategory, Product, Employee, Customer, and InventoryLocation establish the master registries. These entities possess intrinsic business identifiers and maintain independent lifecycles.
- **Weak and Multi-Valued Characteristic Entities:** SupplierPhone and CustomerPhone represent multi-valued phone attributes functionally dependent on their parent owner entity, resolved into distinct relations using composite primary keys.
- **Associative Bridge Entities:** PurchaseOrderLine, ShipmentLine, and SalesTransactionLine resolve many-to-many (M:N) relationships between parent transactions and merchandise products. These entities carry line-specific operational attributes such as contracted unit prices, ordered quantities, shipped versus received quantities, and sold quantities.
- **Dynamic State & Movement Ledgers:** InventoryStock maintains live stock balances per location-product pair. InventoryTransaction maintains an immutable chronological audit trail. RestockEvent and ReorderTrigger capture reactive event logs generated when inventory thresholds are crossed.

5.2 Entity-Relationship (ER) Diagram

The Entity-Relationship (ER) diagram models the logical associations connecting the approved 19 relations. It illustrates 1:N vendor-to-order associations, 1:N order-to-shipment tracking, composite-key associative lines, and the separation of bulk warehouses from retail shelves.

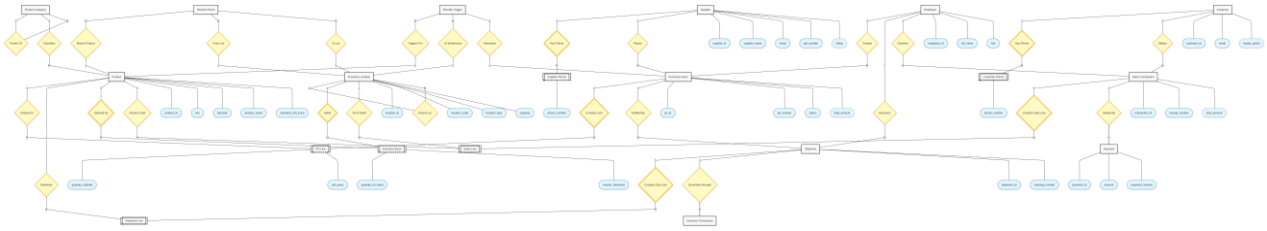


Figure 1 — SmartRetail Entity-Relationship Diagram (ERD)

Comprehensive ER Diagram illustrating the 19 core relations, associative line entities, composite keys, and relational cardinality across procurement, inventory, and POS subsystems.

5.3 Enhanced Entity-Relationship (EER) Diagram

The Enhanced Entity-Relationship (EER) diagram extends the base model by introducing specialization hierarchies, recursive relationships, and category taxonomies:

- **Recursive Hierarchies:** ProductCategory utilizes a self-referencing relationship (parent_category_id -> category_id) to model multi-tier subcategory trees. Similarly, InventoryLocation utilizes parent_location_id -> location_id to establish parent warehouse associations for display shelves.
- **Disjoint Location Specialization:** InventoryLocation enforces a specialization hierarchy partitioning locations into WAREHOUSE and SHELF types via CHECK constraint, dictating permissible operational flows.

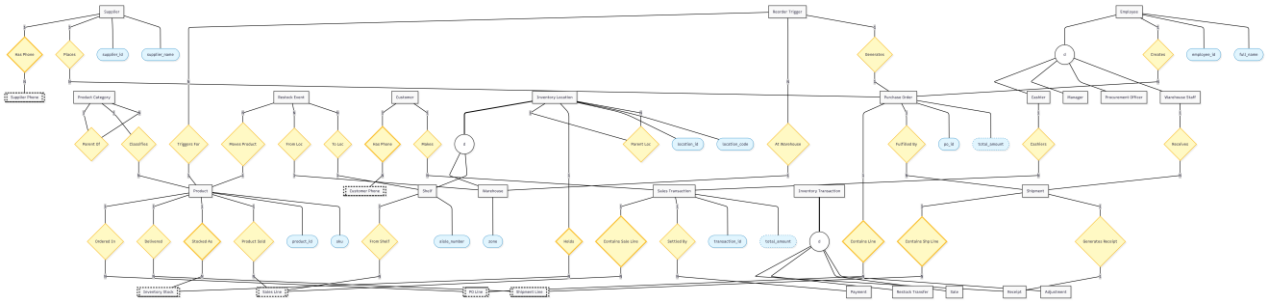


Figure 2 — SmartRetail Enhanced Entity-Relationship Diagram (EERD)

Enhanced ER Diagram depicting specialization of inventory locations (Warehouse vs. Shelf), recursive category taxonomies, and split-tender payment associations.

6. Relational Normalization Analysis

Normalization is the formal mathematical technique utilized during database design to structure relational tables in a manner that eliminates data redundancy, guarantees attribute dependency on candidate keys, and prevents update, insertion, and deletion anomalies.

6.1 Purpose and Necessity of Normalization in SmartRetail

An unnormalized database design introduces severe operational hazards into retail management:

- **Insertion Anomalies:** In an unnormalized design combining products and categories, a new merchandise category cannot be created unless at least one product has been procured and assigned to it. Similarly, a supplier cannot be registered without an associated purchase order.
- **Deletion Anomalies:** If the last product in a particular category is discontinued and deleted, all record of the parent category itself is inadvertently destroyed.
- **Update Anomalies:** If vendor contact details, product names, or category descriptions are repeated across thousands of sales lines or inventory rows, modifying a vendor address requires updating hundreds of records. Any missed update creates data inconsistency where the same entity has contradictory states across the database.
- **Data Redundancy & Storage Inefficiency:** Replicating textual product descriptions, customer profiles, and supplier tax IDs across every sales receipt consumes storage and causes I/O bottlenecks during transactional joins.

6.2 Conceptual Unnormalized Form (UNF)

To demonstrate the normalization process rigorously, consider a hypothetical, conceptual unnormalized operational record (UNF) that an unnormalized retail spreadsheet or legacy flat-file might capture. (Note: This monolithic structure is purely conceptual and never existed in the implemented database).

> **Conceptual Unnormalized Form (UNF) Definition:** A single monolithic record structure containing non-atomic multi-valued fields (e.g., repeating phone numbers) and repeating groups (e.g., multiple item lines per order, multiple shipment receipts per PO, and multiple tender payments per sale).

PO_Num	Supplier	Supplier Phones	Product Lines (SKU, Name, Cat, Qty, Price)	Tracking	Warehouse	Customer	Receipt_No	Split Payments
PO-1001	Acme Foods	{9876543210, 9876543211}	{SKU-01 (Milk, Dairy, 50, 45.00), SKU-04 (Butter, Dairy, 20, 220.00)}	TRK-901	WH-01	CUST-501	RCP-8801	{Cash: 500, Card: 1200}
PO-1002	Nestle Dist	{9123456780}	{SKU-07 (Noodles, Packaged, 100, 15.00)}	TRK-905	WH-02	CUST-502	RCP-8802	{UPI: 450}

6.3 First Normal Form (1NF)

A relation is in First Normal Form (1NF) if and only if:

- **Atomic Domains:** Every attribute contains only atomic (indivisible) values. There are no sets, arrays, or composite lists within a single cell.
- **No Repeating Groups:** There are no repeating columns or multi-valued attribute groupings.
- **Unique Identification:** Each row is uniquely identifiable by a defined Primary Key.

Decomposition to 1NF in SmartRetail:

- **Multi-Valued Phone Numbers:** Suppliers and Customers may possess multiple telephone contact numbers. Rather than storing comma-separated strings or defining flat columns (phone1, phone2), multi-valued phone numbers are extracted into dedicated relations:

• SupplierPhone with composite primary key: (supplier_id, phone_number)
• CustomerPhone with composite primary key: (customer_id, phone_number)

- **Repeating Order Lines:** Multi-item purchase orders and point-of-sale transactions are decomposed into distinct row entities (PurchaseOrderLine, SalesTransactionLine), each holding atomic quantity and price values.

supplier_id (PK)	supplier_name	phone_number (PK)
1	Acme Foods Supply	9876543210
1	Acme Foods Supply	9876543211
2	Heritage Dairy Ltd	9123456780

6.4 Second Normal Form (2NF)

A relation is in Second Normal Form (2NF) if and only if:

- **1NF Compliance:** It is already in First Normal Form.
- **No Partial Dependencies:** Every non-prime attribute is fully functionally dependent on the ENTIRE primary key, rather than on a proper subset of a composite primary key.

In relations with single-attribute primary keys (such as Supplier, Product, Customer, Employee, PurchaseOrder), partial dependency is impossible; hence, they automatically satisfy 2NF once in 1NF. However, relations with composite primary keys require careful partial dependency analysis:

Case Analysis 1: PurchaseOrderLine

Composite Primary Key: (po_id, product_id)

• quantity_ordered depends on BOTH po_id and product_id. (A purchase order specifies different quantities for different products).
• unit_price represents the contracted unit cost for that specific product on that specific order.
• Neither quantity_ordered nor unit_price depends solely on po_id (an order contains many products) nor solely on product_id (a product has different prices across different vendor orders over time).
Therefore, PurchaseOrderLine satisfies 2NF.

Case Analysis 2: ShipmentLine

Composite Primary Key: (shipment_id, product_id)

• quantity_shipped and quantity_received depend strictly on the pair (shipment_id, product_id). An individual shipment contains multiple products, and a product may be distributed across multiple shipments. Hence, full functional dependency is preserved.

Case Analysis 3: InventoryStock

Composite Primary Key: (location_id, product_id)

• quantity_on_hand, reorder_threshold, and reorder_quantity represent the stock balance and threshold for a specific product at a specific physical location. A product has different quantities in Warehouse 1 versus Shelf 3; similarly, a location holds different quantities of different products. Full dependency on (location_id, product_id) is strictly maintained.

Case Analysis 4: SalesTransactionLine

Composite Primary Key: (transaction_id, product_id)

• quantity_sold, unit_price, and shelf_location_id depend on both the sale transaction and the product dispensed. The sold unit price captures historical checkout pricing at the moment of sale, and shelf_location_id identifies the dispensing retail shelf. Both attributes require the complete key.

6.5 Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if and only if:

- **2NF Compliance:** It is already in Second Normal Form.
- **No Transitive Dependencies:** No non-prime attribute is transitively dependent on the primary key (i.e., there is no dependency chain of the form X -> Y -> Z where X is the primary key and Y is a non-prime attribute).

Project-Specific 3NF Decompositions in SmartRetail:

1. Product and ProductCategory Decomposition

In an unnormalized product table, storing category information alongside products creates a transitive dependency:
• product_id -> category_id
• category_id -> category_name, parent_category_id
• Therefore: product_id -> category_name (transitive).
Decomposition: The transitive dependency was eliminated by extracting category details into a separate relation, ProductCategory(category_id, category_name, parent_category_id), leaving only the foreign key category_id in Product.

2. PurchaseOrder and Supplier Decomposition

Storing vendor commercial attributes inside PurchaseOrder creates a transitive dependency:
• po_id -> supplier_id
• supplier_id -> supplier_name, email, gst_number, rating, street, city
• Therefore: po_id -> supplier_name (transitive).
Decomposition: Extracted vendor attributes into Supplier(supplier_id, ...), retaining only supplier_id in PurchaseOrder.

3. SalesTransaction and Customer Decomposition

In checkout transactions, customer metadata introduces transitive dependency:
• transaction_id -> customer_id
• customer_id -> first_name, last_name, email, loyalty_points
• Therefore: transaction_id -> loyalty_points (transitive).
Decomposition: Customer profile attributes are isolated into Customer(customer_id, ...), referencing customer_id optionally in SalesTransaction.

6.6 Boyce-Codd Normal Form (BCNF)

A relation is in Boyce-Codd Normal Form (BCNF) if and only if for every non-trivial functional dependency X -> Y, X is a superkey (either the primary key or an alternate candidate key). BCNF is a stricter version of 3NF that eliminates anomalies resulting from overlapping candidate keys.

Rigorous BCNF Audit across SmartRetail Relations:

- **Relations with Single Candidate Keys:** In relations where the Primary Key is the sole candidate key (e.g., InventoryLocation, InventoryTransaction, RestockEvent, ReorderTrigger, ShipmentLine, PurchaseOrderLine, SalesTransactionLine, Payment), every non-trivial functional dependency has the primary key as its determinant. Since the primary key is a superkey, these relations trivially satisfy BCNF.

- **Relations with Multiple Candidate Keys:** Several relations possess alternate candidate keys enforced via UNIQUE constraints:

• Supplier: Determinants are supplier_id (PK), email (UK), and gst_number (UK). Since each determinant uniquely identifies a vendor record, all determinants are superkeys. BCNF is satisfied.
• Product: Determinants are product_id (PK), sku (UK), and barcode (UK). Each is a unique candidate key capable of identifying the merchandise record. BCNF is satisfied.
• Customer: Determinants are customer_id (PK) and email (UK). Both determinants are superkeys. BCNF is satisfied.
• PurchaseOrder: Determinants are po_id (PK) and po_number (UK). Both are superkeys. BCNF is satisfied.
• SalesTransaction: Determinants are transaction_id (PK) and receipt_number (UK). Both are superkeys. BCNF is satisfied.

6.7 Final 19 Normalized Relations

The resulting 19 relations form a fully normalized, anomaly-free schema:

#	Relation Name	Primary Key	Alternate / Unique Keys	Normalization Justification
1	Supplier	supplier_id	email, gst_number	Master vendor registry; isolated from orders to prevent vendor deletion anomalies.
2	SupplierPhone	(supplier_id, phone_number)	—	1NF decomposition isolating multi-valued phone numbers.
3	ProductCategory	category_id	category_name	3NF decomposition isolating category taxonomy from product catalog.
4	Product	product_id	sku, barcode	Master merchandise catalog; isolated from inventory locations and transactions.
5	Employee	employee_id	email	Store personnel directory with role-based segregation.
6	PurchaseOrder	po_id	po_number	Procurement order headers placed with certified commercial suppliers.
7	PurchaseOrderLine	(po_id, product_id)	—	Associative bridge resolving M:N procurement relationships in 2NF/BCNF.
8	Shipment	shipment_id	tracking_number	Logistics freight tracking isolated from purchase orders.
9	ShipmentLine	(shipment_id, product_id)	—	Receiving reconciliation bridge verifying physical versus shipped quantities.
10	InventoryLocation	location_id	location_code	Physical storage mapping isolating warehouse from display shelf locations.
11	InventoryStock	(location_id, product_id)	—	Live inventory balances and reorder thresholds in 2NF/BCNF.
12	InventoryTransaction	transaction_id	—	Append-only immutable audit ledger recording every stock movement.
13	RestockEvent	restock_id	—	Audit event log tracking internal warehouse-to-shelf transfers.
14	ReorderTrigger	trigger_id	—	Automated alert ledger for depleted warehouse safety thresholds.
15	Customer	customer_id	email	Shopper profiles, loyalty balances, and membership tiers.

#	Relation Name	Primary Key	Alternate / Unique Keys	Normalization Justification
16	CustomerPhone	(customer_id, phone_number)	—	1NF decomposition isolating multi-valued customer telephone contacts.
17	SalesTransaction	transaction_id	receipt_number	POS checkout invoice headers supporting guest shoppers.
18	SalesTransactionLine	(transaction_id, product_id)	—	Associative checkout bridge recording unit price and shelf location in 2NF.
19	Payment	payment_id	—	Multi-tender settlement entity supporting 1:N split payments per sale.

6.8 Normalization Progression Summary

The following progression table summarizes the stepwise transformation of the conceptual model through each normal form stage:

Transformation Stage	Target Anomalies / Flaws	Decomposed Relations	Resulting Structural Guarantee
UNF -> 1NF	Multi-valued phone lists; repeating item groups	SupplierPhone, CustomerPhone, Line items	Attributes made atomic; primary keys defined.
1NF -> 2NF	Composite keys in line items and stock balances	PurchaseOrderLine, ShipmentLine, InventoryStock, SalesTransactionLine	Partial dependencies removed; non-key attributes depend on entire composite key.
2NF -> 3NF	Transitive dependencies (Category in Product, Vendor in PO, Customer in Sale)	ProductCategory, Supplier, Customer isolated	Transitive dependencies eliminated; non-key attributes depend non-transitively on PK.
3NF -> BCNF	Multiple candidate keys in Master and Transaction entities	Supplier (email, gst), Product (sku, barcode), PO (po_number), Sale (receipt_no)	Verified that every determinant is a superkey across all 19 relations.

7. Formal Functional Dependencies Specification

Functional dependencies (FDs) represent mathematical constraints between attributes in a relation. In SmartRetail, functional dependencies are strictly grounded in declared primary keys and unique constraints enforced by Oracle 11g XE. Below is the formal specification of non-trivial dependencies:

Functional Dependencies: Supplier

- supplier_id -> supplier_name, contact_person, email, gst_number, street, city, state, pincode, rating, created_at (Primary Key)
- email -> supplier_id, supplier_name, contact_person, gst_number, street, city, state, pincode, rating, created_at (Candidate Key / UK)
- gst_number -> supplier_id, supplier_name, contact_person, email, street, city, state, pincode, rating, created_at (Candidate Key / UK)

Functional Dependencies: SupplierPhone

- (supplier_id, phone_number) -> Ø (All-key relation; composite key)

Functional Dependencies: ProductCategory

- category_id -> category_name, parent_category_id (Primary Key)
- category_name -> category_id, parent_category_id (Candidate Key / UK)

Functional Dependencies: Product

- product_id -> sku, barcode, product_name, description, category_id, unit_of_measure, standard_unit_price, is_active (Primary Key)
- sku -> product_id, barcode, product_name, description, category_id, unit_of_measure, standard_unit_price, is_active (Candidate Key / UK)
- barcode -> product_id, sku, product_name, description, category_id, unit_of_measure, standard_unit_price, is_active (Candidate Key / UK)

Functional Dependencies: Employee

- employee_id -> full_name, role, email, phone, hire_date (Primary Key)
- email -> employee_id, full_name, role, phone, hire_date (Candidate Key / UK)

Functional Dependencies: PurchaseOrder

- po_id -> po_number, supplier_id, created_by, order_date, expected_delivery_date, status, total_amount, is_auto_generated (Primary Key)
- po_number -> po_id, supplier_id, created_by, order_date, expected_delivery_date, status, total_amount, is_auto_generated (Candidate Key / UK)

Functional Dependencies: PurchaseOrderLine

- (po_id, product_id) -> quantity_ordered, unit_price (Composite Primary Key)

Functional Dependencies: Shipment

- shipment_id -> tracking_number, po_id, carrier, shipment_date, received_date, status, received_by (Primary Key)
- tracking_number -> shipment_id, po_id, carrier, shipment_date, received_date, status, received_by (Candidate Key / UK)

Functional Dependencies: ShipmentLine

- (shipment_id, product_id) -> quantity_shipped, quantity_received (Composite Primary Key)

Functional Dependencies: InventoryLocation

- location_id -> location_code, location_type, location_name, parent_location_id, capacity, aisle_number, zone (Primary Key)
- location_code -> location_id, location_type, location_name, parent_location_id, capacity, aisle_number, zone (Candidate Key / UK)

Functional Dependencies: InventoryStock

- (location_id, product_id) -> quantity_on_hand, reorder_threshold, reorder_quantity, last_updated_at (Composite Primary Key)

Functional Dependencies: InventoryTransaction

- transaction_id -> product_id, location_id, transaction_type, quantity, transaction_date, reference_type, reference_id (Primary Key)

Functional Dependencies: RestockEvent

- restock_id -> product_id, from_location_id, to_location_id, quantity_moved, event_date, triggered_by, performed_by, status (Primary Key)

Functional Dependencies: ReorderTrigger

- trigger_id -> product_id, location_id, stock_at_trigger, threshold_at_trigger, trigger_date, generated_po_id, status (Primary Key)

Functional Dependencies: Customer

- customer_id -> first_name, last_name, email, loyalty_points, membership_type, registered_date (Primary Key)
- email -> customer_id, first_name, last_name, loyalty_points, membership_type, registered_date (Candidate Key / UK)

Functional Dependencies: CustomerPhone

- (customer_id, phone_number) -> Ø (All-key relation; composite key)

Functional Dependencies: SalesTransaction

- transaction_id -> receipt_number, customer_id, cashier_id, transaction_date, total_amount, status (Primary Key)
- receipt_number -> transaction_id, customer_id, cashier_id, transaction_date, total_amount, status (Candidate Key / UK)

Functional Dependencies: SalesTransactionLine

- (transaction_id, product_id) -> shelf_location_id, quantity_sold, unit_price (Composite Primary Key)

Functional Dependencies: Payment

- payment_id -> transaction_id, payment_method, amount, payment_date, reference_number, status (Primary Key)

8. Complete Relational Schema Specification

The SmartRetail database comprises exactly 19 approved relations, partitioned into four operational subsystems. The detailed data dictionaries below define exact attribute data types, nullability rules, key designations, and domain check constraints enforced within Oracle 11g XE.

8.1 Procurement & Supplier Relations Subsystem

This subsystem manages commercial vendor accounts, multi-valued contact channels, formal purchase orders, itemized lines, inbound carrier shipments, and physical receiving reconciliation.

Table 1: Supplier

Column Name	Data Type	Null?	Key	Description & Domain Constraint
supplier_id	NUMBER(10)	No	PK	Primary surrogate vendor identifier (SEQ_SUPPLIER_ID)
supplier_name	VARCHAR2(100)	No	None	Commercial trading name of the supplier
contact_person	VARCHAR2(100)	Yes	None	Primary liaison representative
email	VARCHAR2(150)	No	UK	Corporate communication email (UNIQUE)
gst_number	VARCHAR2(20)	No	UK	Goods and Services Tax identification number (UNIQUE)
street	VARCHAR2(200)	Yes	None	Physical street address
city	VARCHAR2(100)	Yes	None	Postal municipality / city
state	VARCHAR2(100)	Yes	None	Federal state / province
pincode	VARCHAR2(10)	Yes	None	Postal code
rating	NUMBER(2,1)	Yes	CHECK	Quality performance score (rating BETWEEN 1 AND 5)
created_at	TIMESTAMP(6)	Yes	None	Vendor onboarding timestamp (DEFAULT SYSTIMESTAMP)

Table 2: SupplierPhone

Column Name	Data Type	Null?	Key	Description & Domain Constraint
supplier_id	NUMBER(10)	No	PK, FK	Foreign key referencing Supplier(supplier_id)
phone_number	VARCHAR2(20)	No	PK	Telephone contact number in 1NF composite key

Table 3: PurchaseOrder

Column Name	Data Type	Null?	Key	Description & Domain Constraint
po_id	NUMBER(10)	No	PK	Primary surrogate order key (SEQ_PO_ID)
po_number	VARCHAR2(50)	No	UK	Business procurement order number (UNIQUE)
supplier_id	NUMBER(10)	No	FK	Referenced vendor -> Supplier(supplier_id)
created_by	NUMBER(10)	No	FK	Authoring employee -> Employee(employee_id)
order_date	DATE	No	None	Date order was placed (DEFAULT SYSDATE)
expected_delivery_date	DATE	Yes	None	Contracted expected shipment date
status	VARCHAR2(30)	No	CHECK	CHECK: PENDING, APPROVED, PARTIALLY_RECEIVED, RECEIVED, CANCELLED

Column Name	Data Type	Null?	Key	Description & Domain Constraint
total_amount	NUMBER(12,2)	No	CHECK	Total contracted amount (CHECK: total_amount >= 0)
is_auto_generated	VARCHAR2(5)	No	CHECK	Generated by trigger? (CHECK: 'TRUE', 'FALSE')

Table 4: PurchaseOrderLine

Column Name	Data Type	Null?	Key	Description & Domain Constraint
po_id	NUMBER(10)	No	PK, FK	Foreign key referencing PurchaseOrder(po_id)
product_id	NUMBER(10)	No	PK, FK	Foreign key referencing Product(product_id)
quantity_ordered	NUMBER(8)	No	CHECK	Units ordered (CHECK: quantity_ordered > 0)
unit_price	NUMBER(10,2)	No	CHECK	Contracted unit cost (CHECK: unit_price > 0)

Table 5: Shipment

Column Name	Data Type	Null?	Key	Description & Domain Constraint
shipment_id	NUMBER(10)	No	PK	Primary surrogate shipment key (SEQ_SHIPMENT_ID)
tracking_number	VARCHAR2(50)	No	UK	Carrier freight tracking code (UNIQUE)
po_id	NUMBER(10)	No	FK	Referenced purchase order -> PurchaseOrder(po_id)
carrier	VARCHAR2(100)	No	None	Logistics transportation carrier name
shipment_date	DATE	No	None	Dispatch date from vendor
received_date	DATE	Yes	None	Physical warehouse arrival date
status	VARCHAR2(30)	No	CHECK	CHECK: IN_TRANSIT, DELIVERED, CANCELLED
received_by	NUMBER(10)	Yes	FK	Receiving staff member -> Employee(employee_id)

Table 6: ShipmentLine

Column Name	Data Type	Null?	Key	Description & Domain Constraint
shipment_id	NUMBER(10)	No	PK, FK	Foreign key referencing Shipment(shipment_id)
product_id	NUMBER(10)	No	PK, FK	Foreign key referencing Product(product_id)
quantity_shipped	NUMBER(8)	No	CHECK	Shipped count (CHECK: quantity_shipped >= 0)
quantity_received	NUMBER(8)	No	CHECK	Received count (CHECK: quantity_received >= 0)

8.2 Catalog & Store Operations Subsystem

This subsystem defines merchandise taxonomy hierarchies, the master product catalog, retail pricing, and store personnel authorization directories.

Table 7: ProductCategory

Column Name	Data Type	Null?	Key	Description & Domain Constraint
category_id	NUMBER(10)	No	PK	Primary category key (SEQ_CATEGORY_ID)

Column Name	Data Type	Null?	Key	Description & Domain Constraint
category_name	VARCHAR2(100)	No	UK	Unique category title (e.g. Dairy, Beverages)
parent_category_id	NUMBER(10)	Yes	FK	Recursive self-FK -> ProductCategory(category_id)

Table 8: Product

Column Name	Data Type	Null?	Key	Description & Domain Constraint
product_id	NUMBER(10)	No	PK	Primary surrogate product identifier (SEQ_PRODUCT_ID)
sku	VARCHAR2(50)	No	UK	Stock Keeping Unit code (UNIQUE)
barcode	VARCHAR2(50)	No	UK	EAN/UPC retail barcode identifier (UNIQUE)
product_name	VARCHAR2(150)	No	None	Commercial product description
description	VARCHAR2(500)	Yes	None	Detailed specifications and packaging details
category_id	NUMBER(10)	No	FK	Classification foreign key -> ProductCategory
unit_of_measure	VARCHAR2(20)	No	None	Packaging unit (e.g. Litre, Kg, Pack, Bottle)
standard_unit_price	NUMBER(10,2)	No	CHECK	Active retail sales price (CHECK: price > 0)
is_active	VARCHAR2(5)	No	CHECK	Active merchandising flag (CHECK: 'TRUE', 'FALSE')

Table 9: Employee

Column Name	Data Type	Null?	Key	Description & Domain Constraint
employee_id	NUMBER(10)	No	PK	Surrogate personnel identifier (SEQ_EMPLOYEE_ID)
full_name	VARCHAR2(100)	No	None	Legal full name of employee
role	VARCHAR2(30)	No	CHECK	CHECK: MANAGER, CASHIER, INVENTORY_CLERK, ADMIN
email	VARCHAR2(150)	No	UK	Corporate user email (UNIQUE)
phone	VARCHAR2(20)	Yes	None	Primary contact telephone
hire_date	DATE	No	None	Staff onboarding date

8.3 Multi-Tier Inventory & Replenishment Subsystem

This subsystem models physical location segregation, location-specific inventory balances, an immutable stock movement audit ledger, and automated reactive restock and reorder events.

Table 10: InventoryLocation

Column Name	Data Type	Null?	Key	Description & Domain Constraint
location_id	NUMBER(10)	No	PK	Surrogate location identifier (SEQ_LOCATION_ID)
location_code	VARCHAR2(50)	No	UK	Unique facility location code (e.g. WH-CENTRAL-01)
location_type	VARCHAR2(20)	No	CHECK	Physical storage zone (CHECK: WAREHOUSE, SHELF)
location_name	VARCHAR2(100)	No	None	Human-readable facility name
parent_location_id	NUMBER(10)	Yes	FK	Parent facility link -> InventoryLocation(location_id)

Column Name	Data Type	Null?	Key	Description & Domain Constraint
capacity	NUMBER(8)	Yes	CHECK	Volumetric unit capacity (CHECK: capacity > 0)
aisle_number	VARCHAR2(20)	Yes	None	Aisle coordinate (for retail shelves)
zone	VARCHAR2(50)	Yes	None	Operational zone (e.g. Ambient, Cold Chain)

Table 11: InventoryStock

Column Name	Data Type	Null?	Key	Description & Domain Constraint
location_id	NUMBER(10)	No	PK, FK	Storage facility -> InventoryLocation(location_id)
product_id	NUMBER(10)	No	PK, FK	Stored merchandise item -> Product(product_id)
quantity_on_hand	NUMBER(8)	No	CHECK	Current units on hand (CHECK: quantity_on_hand >= 0)
reorder_threshold	NUMBER(8)	No	CHECK	Minimum trigger level (CHECK: reorder_threshold >= 0)
reorder_quantity	NUMBER(8)	No	CHECK	Standard replenish quantity (CHECK: reorder_qty > 0)
last_updated_at	TIMESTAMP(6)	No	None	Timestamp of last stock mutation

Table 12: InventoryTransaction

Column Name	Data Type	Null?	Key	Description & Domain Constraint
transaction_id	NUMBER(10)	No	PK	Primary surrogate audit key (SEQ_ITXN_ID)
product_id	NUMBER(10)	No	FK	Mutated merchandise item -> Product(product_id)
location_id	NUMBER(10)	No	FK	Affected facility -> InventoryLocation(location_id)
transaction_type	VARCHAR2(30)	No	CHECK	CHECK: RECEIPT, TRANSFER_IN, TRANSFER_OUT, SALE, ADJUSTMENT
quantity	NUMBER(8)	No	CHECK	Absolute quantity moved (CHECK: quantity > 0)
transaction_date	TIMESTAMP(6)	No	None	Audit event timestamp (DEFAULT SYSTIMESTAMP)
reference_type	VARCHAR2(30)	Yes	CHECK	CHECK: SHIPMENT, RESTOCK, SALE, MANUAL
reference_id	NUMBER(10)	Yes	None	Foreign operational entity ID (e.g. sale_id, shipment_id)

Table 13: RestockEvent

Column Name	Data Type	Null?	Key	Description & Domain Constraint
restock_id	NUMBER(10)	No	PK	Primary surrogate restock key (SEQ_RESTOCK_ID)
product_id	NUMBER(10)	No	FK	Replenished product -> Product(product_id)
from_location_id	NUMBER(10)	No	FK	Source warehouse -> InventoryLocation(location_id)
to_location_id	NUMBER(10)	No	FK	Target retail shelf -> InventoryLocation(location_id)
quantity_moved	NUMBER(8)	No	CHECK	Replenished units (CHECK: quantity_moved > 0)
event_date	TIMESTAMP(6)	No	None	Trigger execution timestamp
triggered_by	VARCHAR2(50)	No	None	Mechanism (AUTO_THRESHOLD or Staff Name)

Column Name	Data Type	Null?	Key	Description & Domain Constraint
performed_by	NUMBER(10)	Yes	FK	Warehouse personnel executing transfer -> Employee
status	VARCHAR2(30)	No	CHECK	CHECK: PENDING, COMPLETED

Table 14: ReorderTrigger

Column Name	Data Type	Null?	Key	Description & Domain Constraint
trigger_id	NUMBER(10)	No	PK	Primary surrogate trigger key (SEQ_TRIGGER_ID)
product_id	NUMBER(10)	No	FK	Depleted product -> Product(product_id)
location_id	NUMBER(10)	No	FK	Breached warehouse -> InventoryLocation(location_id)
stock_at_trigger	NUMBER(8)	No	None	Actual quantity on hand when trigger fired
threshold_at_trigger	NUMBER(8)	No	None	Target safety threshold breached
trigger_date	TIMESTAMP(6)	No	None	Trigger activation timestamp
generated_po_id	NUMBER(10)	Yes	FK	Automated purchase order link -> PurchaseOrder(po_id)
status	VARCHAR2(30)	No	CHECK	CHECK: OPEN, PO_CREATED, RESOLVED, IGNORED

8.4 Point of Sale (POS) & Customer Billing Subsystem

This subsystem captures customer account profiles, multi-valued telephone communication, POS invoices, itemized checkout lines with shelf attribution, and split-tender payment settlement.

Table 15: Customer

Column Name	Data Type	Null?	Key	Description & Domain Constraint
customer_id	NUMBER(10)	No	PK	Surrogate customer identifier (SEQ_CUSTOMER_ID)
first_name	VARCHAR2(100)	No	None	Customer given name
last_name	VARCHAR2(100)	No	None	Customer surname / family name
email	VARCHAR2(150)	No	UK	Unique customer account email (UNIQUE)
loyalty_points	NUMBER(8)	No	CHECK	Accumulated loyalty score (CHECK: points >= 0)
membership_type	VARCHAR2(30)	No	CHECK	CHECK: REGULAR, SILVER, GOLD, PLATINUM
registered_date	DATE	No	None	Account creation date (DEFAULT SYSDATE)

Table 16: CustomerPhone

Column Name	Data Type	Null?	Key	Description & Domain Constraint
customer_id	NUMBER(10)	No	PK, FK	Foreign key referencing Customer(customer_id)
phone_number	VARCHAR2(20)	No	PK	Telephone contact number in 1NF composite key

Table 17: SalesTransaction

Column Name	Data Type	Null?	Key	Description & Domain Constraint
transaction_id	NUMBER(10)	No	PK	Primary surrogate sale key (SEQ_SALES_ID)
receipt_number	VARCHAR2(50)	No	UK	Fiscal printed receipt number (UNIQUE)
customer_id	NUMBER(10)	Yes	FK	Shopper account (NULL for anonymous guest shoppers)
cashier_id	NUMBER(10)	No	FK	Cashier personnel -> Employee(employee_id)
transaction_date	TIMESTAMP(6)	No	None	Invoice timestamp (DEFAULT SYSTIMESTAMP)
total_amount	NUMBER(10,2)	No	CHECK	Synchronized transaction total (CHECK: total >= 0)
status	VARCHAR2(20)	No	CHECK	CHECK: COMPLETED, VOID, REFUNDED

Table 18: SalesTransactionLine

Column Name	Data Type	Null?	Key	Description & Domain Constraint
transaction_id	NUMBER(10)	No	PK, FK	Foreign key referencing SalesTransaction(transaction_id)
product_id	NUMBER(10)	No	PK, FK	Dispensed merchandise -> Product(product_id)
shelf_location_id	NUMBER(10)	No	FK	Dispensing retail shelf -> InventoryLocation
quantity_sold	NUMBER(8)	No	CHECK	Units sold at checkout (CHECK: quantity_sold > 0)
unit_price	NUMBER(10,2)	No	CHECK	Billed unit price at checkout (CHECK: unit_price > 0)

Table 19: Payment

Column Name	Data Type	Null?	Key	Description & Domain Constraint
payment_id	NUMBER(10)	No	PK	Primary surrogate payment identifier (SEQ_PAYMENT_ID)
transaction_id	NUMBER(10)	No	FK	Settled invoice -> SalesTransaction(transaction_id)
payment_method	VARCHAR2(30)	No	CHECK	CHECK: CASH, CREDIT_CARD, DEBIT_CARD, UPI, WALLET, POINTS
amount	NUMBER(10,2)	No	CHECK	Tender amount tendered (CHECK: amount > 0)
payment_date	TIMESTAMP(6)	No	None	Settlement timestamp (DEFAULT SYSTIMESTAMP)
reference_number	VARCHAR2(100)	Yes	None	Electronic transaction auth code / reference
status	VARCHAR2(20)	No	CHECK	CHECK: SUCCESS, PENDING, FAILED, REFUNDED

9. SQL Implementation & Empirical Verification

The SmartRetail relational schema was implemented through modular DDL scripts, populated with representative supermarket seed data, and queried using comprehensive join, aggregation, and analytical SQL statements. Every operation was verified against Oracle Database 11g XE.

9.1 Data Definition Language (DDL) & Schema Generation

Database tables were instantiated via sql/01_schema.sql. Primary keys use Oracle SEQUENCE objects paired with BEFORE INSERT triggers utilizing WHEN (NEW.<col> IS NULL) clauses. This enables deterministic ID allocation during sample seeding while supporting transparent auto-increment surrogate keys during runtime transactions.

```
-- Sequence and transparent trigger definition for Product
CREATE SEQUENCE SEQ_PRODUCT_ID START WITH 1 INCREMENT BY 1 NOCACHE;

CREATE OR REPLACE TRIGGER TRG_BI_PRODUCT
BEFORE INSERT ON Product
FOR EACH ROW
WHEN (NEW.product_id IS NULL)
BEGIN
    SELECT SEQ_PRODUCT_ID.NEXTVAL INTO :NEW.product_id FROM DUAL;
END;
/
```

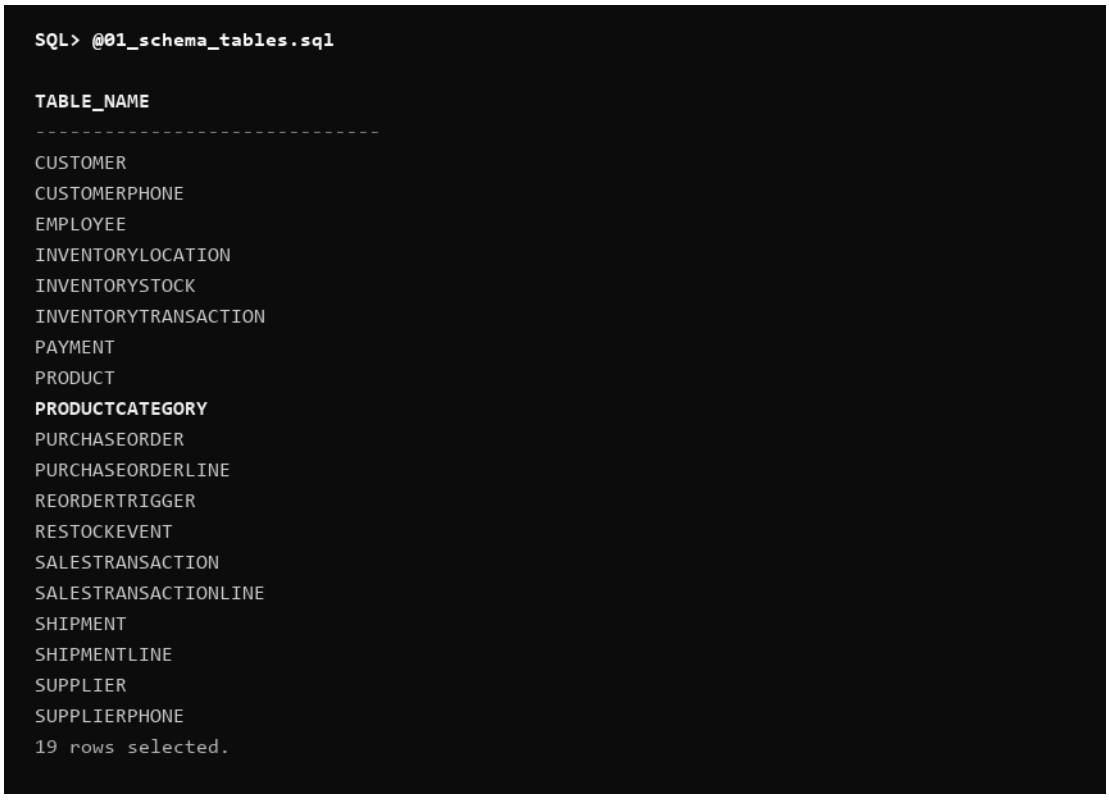


Figure 3 — SmartRetail 19-Relation Schema Verification (SQL*Plus USER_TABLES Query)

*Empirical SQL*Plus execution output querying USER_TABLES, confirming all 19 approved relations are compiled and active under the SYSTEM project schema.*

9.2 Data Population & Catalog Verification

Realistic supermarket merchandise records were loaded via sql/03_sample_data.sql, populating products across Dairy, Bakery, Beverages, Packaged Foods, and Personal Care categories with realistic SKUs, barcodes, and standard prices.

```
SQL> @02_product_data.sql

   ID SKU          BARCODE        PRODUCT_NAME                  CAT UOM           PRICE ACTIVE
----- ------------ -------------- ---------------------------- ----- ---------- -------- ------
    1 SKU-DRY-001  8901030010011  Heritage Full Cream Milk 1L      6 LITER         64.00 TRUE
    2 SKU-DRY-002  8901030010028  Amul Salted Butter 500g          7 PACK         275.00 TRUE
    3 SKU-DRY-003  8901030010035  Amul Processed Cheese Blocks     7 PACK         310.00 TRUE
    4 SKU-BAK-001  8901030020010  Britannia Whole Wheat Bread      8 PACK          50.00 TRUE
    5 SKU-BAK-002  8901030020027  Britannia Good Day Butter Co     9 PACK          45.00 TRUE
    6 SKU-BAK-003  8901030020034  Bourbon Chocolate Cream Bisc     9 PACK          35.00 TRUE
    7 SKU-BEV-001  8901030030019  Real Fruit Power Mixed Fruit    10 TETRAPACK    120.00 TRUE
    8 SKU-BEV-002  8901030030026  Tropicana 100 Orange Juice 1    10 TETRAPACK    145.00 TRUE
    9 SKU-BEV-003  8901030030033  Tata Tea Gold Leaf 500g         11 PACK         320.00 TRUE
   10 SKU-BEV-004  8901030030040  Nescafe Classic Instant Coff    11 JAR          460.00 TRUE
   11 SKU-PER-001  8901030040018  Godrej Cinthol Original Soap     4 PACK         140.00 TRUE
   12 SKU-PER-002  8901030040025  Godrej Protekt Germ Liquid H     4 BOTTLE        99.00 TRUE
   13 SKU-STP-001  8901030050017  Organic Harvest Sona Masoori     5 BAG          390.00 TRUE
   14 SKU-STP-002  8901030050024  Organic Harvest Whole Sharba     5 BAG          285.00 TRUE
   15 SKU-STP-003  8901030050031  Organic Toor Dal Unpolished      5 PACK         175.00 TRUE
15 rows selected.
```

Figure 4 — Product Master Relation Sample Records (SQL*Plus Formatted Output)

Formatted query output from Product, demonstrating master SKUs, barcodes, active retail unit pricing, and merchandise category foreign key assignments.

9.3 Referential and Domain Integrity Constraints

Integrity constraints were compiled via sql/02_constraints.sql, establishing foreign key cascading actions, unique key indexes on business identifiers, and check constraints enforcing positive quantities, prices, and enumerated domains.

```
SQL> @14_constraints.sql


CONSTRAINT_TYPE           ACTIVE_CONSTRAINTS
------------------------- ------------------
CHECK / NOT NULL (CK/NN)                 120
FOREIGN KEY (FK)                          30
PRIMARY KEY (PK)                          19
UNIQUE KEY (UK)                           11

4 rows selected.

================================================================================
SAMPLE OF ACTIVE EXPLICIT BUSINESS CONSTRAINTS
================================================================================

TABLE_NAME          CONSTRAINT_NAME             TYPE
------------------- --------------------------- -----
INVENTORYLOCATION   CK_LOCATION_TYPE            C
INVENTORYLOCATION   CK_LOC_CAPACITY             C
INVENTORYLOCATION   FK_LOC_PARENT               R
INVENTORYLOCATION   PK_INVENTORY_LOCATION       P
INVENTORYLOCATION   UK_LOCATION_CODE            U
INVENTORYSTOCK      CK_STOCK_QOH                C
INVENTORYSTOCK      CK_STOCK_REORDER_QTY        C
INVENTORYSTOCK      CK_STOCK_THRESHOLD          C
INVENTORYSTOCK      FK_STOCK_LOCATION           R
INVENTORYSTOCK      FK_STOCK_PRODUCT            R
INVENTORYSTOCK      PK_INVENTORY_STOCK          P
PRODUCT             CK_PRODUCT_ACTIVE           C
PRODUCT             CK_PRODUCT_PRICE            C
PRODUCT             FK_PROD_CATEGORY            R
PRODUCT             PK_PRODUCT                  P
PRODUCT             UK_PRODUCT_BARCODE          U
```

Figure 5 — Relational Integrity Constraints Enforcement Summary

Data dictionary query on USER_CONSTRAINTS demonstrating active Primary (P), Foreign (R), Unique (U), and Check (C) constraints across all 19 relations.

9.4 Relational Joins & Multi-Location Stock Balances

To verify the multi-tier inventory model, sql/05_joins_aggregations.sql executes relational inner joins connecting InventoryStock with InventoryLocation, grouping stock across bulk warehouses and display shelves.

```
-- Multi-Location Stock Balance Query
SELECT loc.location_code, loc.location_type, p.product_name,
       stk.quantity_on_hand, stk.reorder_threshold
FROM InventoryStock stk
JOIN InventoryLocation loc ON stk.location_id = loc.location_id
JOIN Product p ON stk.product_id = p.product_id
WHERE ROWNUM <= 8;
```

```
SQL> @03_inventory_data.sql

LOC_CODE       LOC_TYPE   SKU          PRODUCT_NAME                 QOH THRESH REORD_QTY
-------------- ---------- ------------ ---------------------------- ----- ------ ---------
WH-CENTRAL-01  WAREHOUSE  SKU-BAK-001  Britannia Whole Wheat Bread    120     30        80
WH-COLD-01     WAREHOUSE  SKU-DRY-001  Heritage Full Cream Milk 1L    180     50       100
WH-COLD-01     WAREHOUSE  SKU-DRY-002  Amul Salted Butter 500g         30     10        25
WH-COLD-01     WAREHOUSE  SKU-DRY-003  Amul Processed Cheese Blocks    18      5        15
SH-DAIRY-01    SHELF      SKU-DRY-001  Heritage Full Cream Milk 1L      8     15        30
SH-DAIRY-01    SHELF      SKU-DRY-002  Amul Salted Butter 500g         16      5        15
SH-DAIRY-01    SHELF      SKU-DRY-003  Amul Processed Cheese Blocks    10      4        10
SH-BAKERY-01   SHELF      SKU-BAK-001  Britannia Whole Wheat Bread      5     10        40
8 rows selected.
```

Figure 6 — Multi-Location Inventory Balances (JOIN InventoryLocation and InventoryStock)

Relational join output displaying location codes, storage types (WAREHOUSE vs. SHELF), product names, live stock balances, and threshold levels.

9.5 Multi-Table POS Receipt Reconstruction Join

A complex 5-table relational join reconstructs complete point-of-sale invoices, joining SalesTransaction, SalesTransactionLine, Product, ProductCategory, and dispensing retail shelf InventoryLocation.

```
-- 5-Table POS Receipt Detail Reconstruction Join
SELECT st.receipt_number, p.product_name, cat.category_name,
       loc.location_code AS shelf_code,
       stl.quantity_sold, stl.unit_price, (stl.quantity_sold * stl.unit_price) AS line_total
FROM SalesTransaction st
JOIN SalesTransactionLine stl ON st.transaction_id = stl.transaction_id
JOIN Product p ON stl.product_id = p.product_id
JOIN ProductCategory cat ON p.category_id = cat.category_id
JOIN InventoryLocation loc ON stl.shelf_location_id = loc.location_id
WHERE ROWNUM <= 6;
```

```
SQL> @04_multi_table_join.sql

RECEIPT_NO        DATE       CASHIER             PRODUCT                 CATEGORY         SHELF          QTY    PRICE     TOTAL
----------------- ---------- ------------------- ----------------------- ---------------- -------------- ---- -------- ---------
RCP-2026-0001     2026-02-15 Divya Nair          Heritage Full Cream Milk Fresh Milk & Cre SH-DAIRY-01       2    64.00    128.00
RCP-2026-0001     2026-02-15 Divya Nair          Amul Salted Butter 500g  Butter & Cheese  SH-DAIRY-01       1   275.00    275.00
RCP-2026-0002     2026-02-15 Karthik Subramanian Britannia Whole Wheat Br Artisan Breads   SH-BAKERY-01      1    50.00     50.00
RCP-2026-0003     2026-02-16 Divya Nair          Organic Harvest Sona Mas Staples & Grains SH-STAPLES-01     1   390.00    390.00
RCP-TEST-PROC-01  2026-09-17 Divya Nair          Amul Salted Butter 500g  Butter & Cheese  SH-DAIRY-01       1   275.00    275.00
RCP-TEST-SUITE-05 2026-09-17 Divya Nair          Britannia Whole Wheat Br Artisan Breads   SH-BAKERY-01      2    50.00    100.00
6 rows selected.
```

Figure 7 — Multi-Table POS Receipt Reconstruction Join (5-Table Join Output)

*SQL*Plus output demonstrating itemized invoice lines, product titles, category names, shelf dispensing locations, and line-item totals.*

9.6 Aggregations with GROUP BY and HAVING

Analytical aggregation queries in sql/05_joins_aggregations.sql evaluate merchandise categories, computing total product counts, average prices, and filtering for high-value merchandise groups using HAVING clauses.

```
SQL> @05_aggregation.sql

CATEGORY             COUNT MIN_PRICE MAX_PRICE AVG_PRICE CATALOG_SUM
-------------------- ----- --------- --------- --------- -----------
Staples & Grains         3    175.00    390.00    283.33      850.00
Butter & Cheese          2    275.00    310.00    292.50      585.00
Cookies & Biscuits       2     35.00     45.00     40.00       80.00
Packaged Juices          2    120.00    145.00    132.50      265.00
Personal Care & Hygi     2     99.00    140.00    119.50      239.00
Tea & Coffee             2    320.00    460.00    390.00      780.00
Artisan Breads           1     50.00     50.00     50.00       50.00
Fresh Milk & Cream       1     64.00     64.00     64.00       64.00
8 rows selected.
```

Figure 8 — Category-Level Sales and Stock Aggregations (GROUP BY & HAVING)

Aggregation query evaluating categories with active product lines, displaying total inventory value, average retail unit prices, and filtered category summaries.

9.7 Advanced SQL & Analytic Window Functions

Advanced queries in sql/06_subqueries_advanced.sql leverage correlated subqueries, Common Table Expressions (WITH clauses), and analytic window functions (RANK() and DENSE_RANK() OVER (PARTITION BY category_id ORDER BY standard_unit_price DESC)) to evaluate pricing rankings within category partitions without self-joins.

```
SQL> @06_advanced_sql.sql

CATEGORY           PRODUCT                                    PRICE  RANK  DENSE_RANK
------------------ ---------------------------------------- -------- ----- ----------
Artisan Breads     Britannia Whole Wheat Bread 400g           50.00     1          1
Butter & Cheese    Amul Processed Cheese Blocks 400g         310.00     1          1
Butter & Cheese    Amul Salted Butter 500g                   275.00     2          2
Cookies & Biscuits Britannia Good Day Butter Cookies 250g     45.00     1          1
Cookies & Biscuits Bourbon Chocolate Cream Biscuits 150g      35.00     2          2
Fresh Milk & Cream Heritage Full Cream Milk 1L                64.00     1          1
Packaged Juices    Tropicana 100 Orange Juice 1L             145.00     1          1
Packaged Juices    Real Fruit Power Mixed Fruit Juice 1L     120.00     2          2
8 rows selected.
```

Figure 9 — Analytic Window Functions: Product Price Rankings by Category Partition

*SQL*Plus output demonstrating analytic RANK() and DENSE_RANK() window functions evaluating retail product price hierarchy partitioned by category.*

9.8 Approved Business Views

Five business views compiled in sql/07_views.sql simplify operational reporting. Specifically, vw_low_stock identifies retail shelves that have breached reorder thresholds, and vw_product_stock provides aggregated inventory counts across all facility zones.

```
SQL> @07_views.sql

========================================================================
VIEW 1: VW_LOW_STOCK (Live Critical Shelf Stock Below Reorder Threshold)
========================================================================
LOC_CODE       TYPE       SKU          PRODUCT_NAME                  QOH THRESH DEFICIT
-------------- ---------- ------------ -------------------------- ----- ------ -------
SH-DAIRY-01    SHELF      SKU-DRY-001  Heritage Full Cream Milk 1L      8     15       7
SH-BAKERY-01   SHELF      SKU-BAK-001  Britannia Whole Wheat Bread      5     10       5
SH-BAKERY-01   SHELF      SKU-BAK-002  Britannia Good Day Butter Co     4     12       8
WH-CENTRAL-01  WAREHOUSE  SKU-BEV-003  Tata Tea Gold Leaf 500g         25     30       5
4 rows selected.
========================================================================
VIEW 2: VW_PRODUCT_STOCK (Enterprise Inventory Valuation Snapshot)
========================================================================
SKU          PRODUCT_NAME                 CATEGORY           WH_QTY SH_QTY TOT_QTY  TOTAL_VAL
------------ ---------------------------- ------------------ ------ ------ ------- ----------
SKU-DRY-001  Heritage Full Cream Milk 1L  Fresh Milk & Cream    180      8     188   12032.00
SKU-DRY-002  Amul Salted Butter 500g      Butter & Cheese        30     16      46   12650.00
SKU-DRY-003  Amul Processed Cheese Blocks Butter & Cheese        18     10      28    8680.00
SKU-BAK-001  Britannia Whole Wheat Bread  Artisan Breads        120      5     125    6250.00
SKU-BAK-002  Britannia Good Day Butter Co Cookies & Biscuits     60      4      64    2880.00
SKU-BAK-003  Bourbon Chocolate Cream Bisc Cookies & Biscuits      0      0       0       0.00
6 rows selected.
```

Figure 10 — Business Views Demonstration: VW_LOW_STOCK and VW_PRODUCT_STOCK

Empirical query output from database views VW_LOW_STOCK and VW_PRODUCT_STOCK demonstrating real-time shelf replenishment alerting.

10. PL/SQL Procedural Implementation

While declarative SQL establishes schema structures and relational constraints, procedural business logic requires an imperative execution framework. SmartRetail implements PL/SQL stored procedures, stored functions, explicit cursors, custom exception handlers, database triggers, and transaction controls to enforce atomic business processes directly within Oracle Database 11g XE.

10.1 Stored Procedures & Business Logic Encapsulation

Stored procedures in SmartRetail encapsulate multi-table write transactions, guaranteeing that physical inventory modifications, financial balances, and movement ledgers are committed or aborted as single atomic units.

Procedure Name	Parameter Signature	Operational Business Logic & Transaction Boundary
PROCESS_SALE	p_cashier_id, p_customer_id, p_payment_method, p_product_id, p_quantity, p_shelf_location_id, p_transaction_id OUT	Validates shelf inventory via SELECT FOR UPDATE. If available stock >= requested quantity, generates SalesTransaction header, appends SalesTransactionLine, records Payment, decrements InventoryStock, and logs an immutable 'SALE' ledger record in InventoryTransaction.
RECEIVE_SHIPMENT	p_shipment_id, p_warehouse_id	Processes incoming supplier carrier deliveries. For each item in ShipmentLine, increments warehouse InventoryStock, logs a 'RECEIPT' movement ledger row, transitions Shipment status to 'RECEIVED', and reconciles parent PurchaseOrder fulfillment.
TRANSFER_STOCK	p_product_id, p_source_location_id, p_target_location_id, p_quantity	Orchestrates internal stock movement from backroom warehouse to retail shelf (or inter-facility transfer). Validates source inventory, decrements source location, increments target location, and appends dual 'TRANSFER_OUT' and 'TRANSFER_IN' audit ledger entries.
CREATE_PURCHASE_ORDER	p_supplier_id, p_created_by, p_product_id, p_quantity, p_unit_price, p_po_id OUT	Authorizes new commercial supplier orders. Generates unique order number, inserts header into PurchaseOrder, appends item line into PurchaseOrderLine, and initializes status to 'PENDING'.

The PROCESS_SALE procedure enforces strict inventory validation before authorizing retail transactions, utilizing row-level locks to prevent race conditions during concurrent sales.

```
-- Procedural Stock Validation in PROCESS_SALE
SELECT quantity_on_hand INTO v_available_stock
FROM InventoryStock
WHERE location_id = p_shelf_location_id AND product_id = p_product_id
FOR UPDATE;

IF v_available_stock < p_quantity THEN
    RAISE_APPLICATION_ERROR(-20012,
        'Insufficient shelf stock. Available: ' || v_available_stock ||
        ', Requested: ' || p_quantity);
END IF;
```

```
SQL> @08_procedures.sql

Savepoint created.
============================================================
PL/SQL STORED PROCEDURES EXECUTION
============================================================
PROCESS_SALE SUCCESS: TxnID=47, Receipt=RCP-EVID-DEMO, Units Sold=2, Total=100
1. PROCESS_SALE: Completed (TxnID=47, Total=100)
RECEIVE_SHIPMENT SUCCESS: ShipmentID=9999, PO=9999, Products Received=1, Target Warehouse=1
2. RECEIVE_SHIPMENT: Completed (Shipment 9999 Received into WH-MAIN-01)
TRANSFER_STOCK SUCCESS: RestockID=14, Product=4, Moved=2 from Loc 1 to Loc 4
3. TRANSFER_STOCK: Completed (2 units moved from WH-MAIN-01 to SH-BAKERY-01)
CREATE_PURCHASE_ORDER SUCCESS: PO_ID=29, PO_Number=PO-EVID-DEMO, Line Total=800, AutoGenerated=FALSE
4. CREATE_PURCHASE_ORDER: Completed (PO_ID=29, 20 units @ 40.00)
============================================================
ALL 4 PROCEDURES EXECUTED SUCCESSFULLY
============================================================
[PASS] PROCESS_SALE
[PASS] RECEIVE_SHIPMENT
[PASS] TRANSFER_STOCK
[PASS] CREATE_PURCHASE_ORDER
PL/SQL procedure successfully completed.
Rollback complete.
```

Figure 11 — PL/SQL Stored Procedures Execution (PROCESS_SALE and RECEIVE_SHIPMENT)

*Empirical execution of stored procedures in SQL*Plus, demonstrating successful retail checkout processing, stock deduction, and inbound shipment receiving.*

10.2 Stored Functions & Financial Calculations

PL/SQL stored functions provide reusable, deterministic computation for financial aggregation, line totals, and valuation metrics across the database.

Function Name	Input Parameters	Return Type	Computational Logic
CALCULATE_SALE_TOTAL	p_transaction_id NUMBER	NUMBER(12,2)	Calculates the gross monetary total for a point-of-sale invoice by summing (quantity_sold * unit_price) across all SalesTransactionLine items belonging to the transaction.
CALCULATE_STOCK_VALUE	p_location_id NUMBER	NUMBER(14,2)	Computes total inventory valuation for a specific facility zone (warehouse or shelf) by joining InventoryStock with Product and aggregating (quantity_on_hand * standard_unit_price).

> **Note on Conceptual vs. Implemented Stored Functions:** During conceptual design (DA1), an auxiliary function CALCULATE_LOYALTY_POINTS was proposed (computing loyalty points as FLOOR(total / 100)). In the approved Oracle 11g XE database implementation, financial calculations are handled by CALCULATE_SALE_TOTAL and CALCULATE_STOCK_VALUE, while customer loyalty points are maintained transactionally via table-level triggers and update operations.

```
SQL> @09_functions.sql

============================================================
FUNCTION 1: CALCULATE_SALE_TOTAL (Approved Business Rule)
============================================================
 TXN_ID CALCULATED_TOTAL
------- ----------------
      1           403.00
      2           165.00
      3          1140.00
      4           275.00
4 rows selected.
============================================================
FUNCTION 2: CALCULATE_STOCK_VALUE (Approved Business Rule)
============================================================
PROD_ID PRODUCT_NAME                     CALCULATED_STOCK_VAL
------- -------------------------------- --------------------
      1 Heritage Full Cream Milk 1L                 12032.00
      2 Amul Salted Butter 500g                     12650.00
      3 Amul Processed Cheese Blocks 400             8680.00
      4 Britannia Whole Wheat Bread 400g             6250.00
4 rows selected.
============================================================
NOTE: CALCULATE_LOYALTY_POINTS is OPTIONAL / DEMONSTRATION ONLY
(No loyalty formula is an approved business rule in DA2 brief)
============================================================
```

Figure 12 — PL/SQL Stored Functions Invocation and Return Verification

*SQL*Plus invocation of stored functions verifying financial valuation computations for invoices and warehouse storage facilities.*

10.3 PL/SQL Cursors: Multi-Row Processing Paradigms

SmartRetail demonstrates three distinct PL/SQL cursor mechanisms in sql/11_cursors.sql for multi-row data processing, auditing, and report generation:

- **Explicit Cursor (OPEN / FETCH / CLOSE):** Demonstrated in low-stock replenishment scans where fine-grained control over fetch batching and %NOTFOUND condition checking is required.
- **Cursor FOR Loop:** Utilized for automated catalog traversal, where Oracle manages cursor allocation, iteration, and termination automatically, minimizing boilerplate code.
- **Parameterized Cursor:** Accepts location or category arguments at invocation (e.g., c_stock_by_loc(p_loc_id NUMBER)), allowing the same compiled cursor logic to be reused across disparate warehouse zones.

```
SQL> @10_cursors.sql

============================================================
1. EXPLICIT CURSOR (OPEN, FETCH, %NOTFOUND, CLOSE)
============================================================
PRODUCT NAME                LOCATION         QOH  THRESH
------------------------------------------------------------
Britannia Good Day Butter   SH-BAKERY-01       4      12
Britannia Whole Wheat Brea  SH-BAKERY-01       5      10
Heritage Full Cream Milk 1  SH-DAIRY-01        8      15
PL/SQL procedure successfully completed.
============================================================
2. CURSOR FOR LOOP (AUTOMATIC OPEN, FETCH, CLOSE)
============================================================
RECEIPT NO        CASHIER              TOTAL (INR)
------------------------------------------------
RCP-2026-0001     Divya Nair               403.00
RCP-2026-0002     Karthik Subramania       165.00
RCP-2026-0003     Divya Nair              1140.00
PL/SQL procedure successfully completed.
============================================================
3. PARAMETERIZED CURSOR (DYNAMIC CATEGORY FILTER)
============================================================
Products in Category 5 (Pantry Staples):
SKU           PRODUCT NAME                    PRICE
------------------------------------------------
SKU-STP-001   Organic Harvest Sona Masoo      390.00
SKU-STP-002   Organic Harvest Whole Shar      285.00
SKU-STP-003   Organic Toor Dal Unpolishe      175.00
PL/SQL procedure successfully completed.
```

Figure 13 — PL/SQL Cursors Demonstration (Explicit, FOR Loop, Parameterized)

*SQL*Plus output demonstrating cursor management patterns: explicit cursor batching, cursor FOR loop catalog iteration, and parameterized cursor location audits.*

10.4 Exception Handling: Predefined & User-Defined Handlers

Robust exception handling ensures that unexpected operational conditions do not corrupt database state or leave open transaction locks. SmartRetail incorporates both predefined Oracle system exceptions and business-specific user-defined exceptions:

- **Predefined System Exceptions:** Handled systematically in procedural blocks, including NO_DATA_FOUND (trapped when querying non-existent product SKUs or barcodes), DUP_VAL_ON_INDEX (trapped when encountering duplicate business keys such as tax GST or email), and ZERO_DIVIDE.
- **User-Defined Business Exceptions:** Custom exceptions mapped using RAISE_APPLICATION_ERROR to specific Oracle error codes in the -20000 to -20999 range. Crucially, error code -20012 is assigned to ex_insufficient_stock, raised whenever requested point-of-sale or transfer quantities exceed available stock on hand.

```
-- Trapping and Raising Business Exceptions
DECLARE
    ex_insufficient_stock EXCEPTION;
    PRAGMA EXCEPTION_INIT(ex_insufficient_stock, -20012);
BEGIN
    PROCESS_SALE(p_cashier_id => 101, p_customer_id => 501,
                 p_payment_method => 'CASH', p_product_id => 10,
                 p_quantity => 9999, p_shelf_location_id => 5,
                 p_transaction_id => v_tx_id);
```

```
EXCEPTION
    WHEN ex_insufficient_stock THEN
        DBMS_OUTPUT.PUT_LINE('Caught Expected Business Exception: ORA-20012');
        ROLLBACK;
END;
```

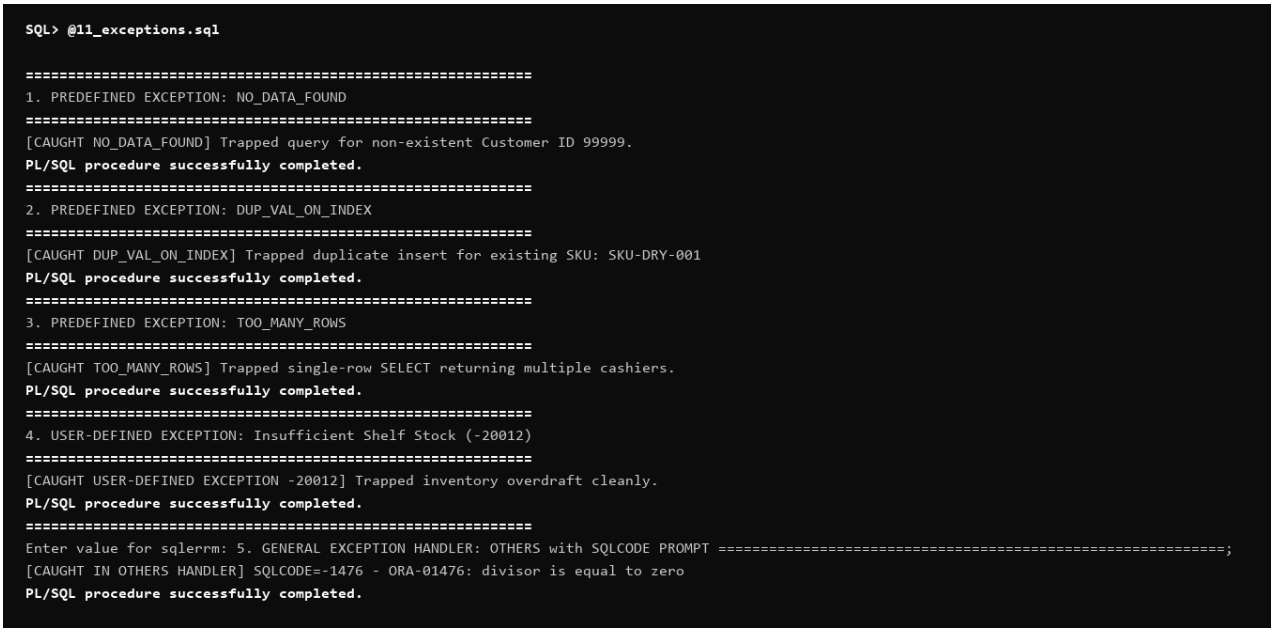


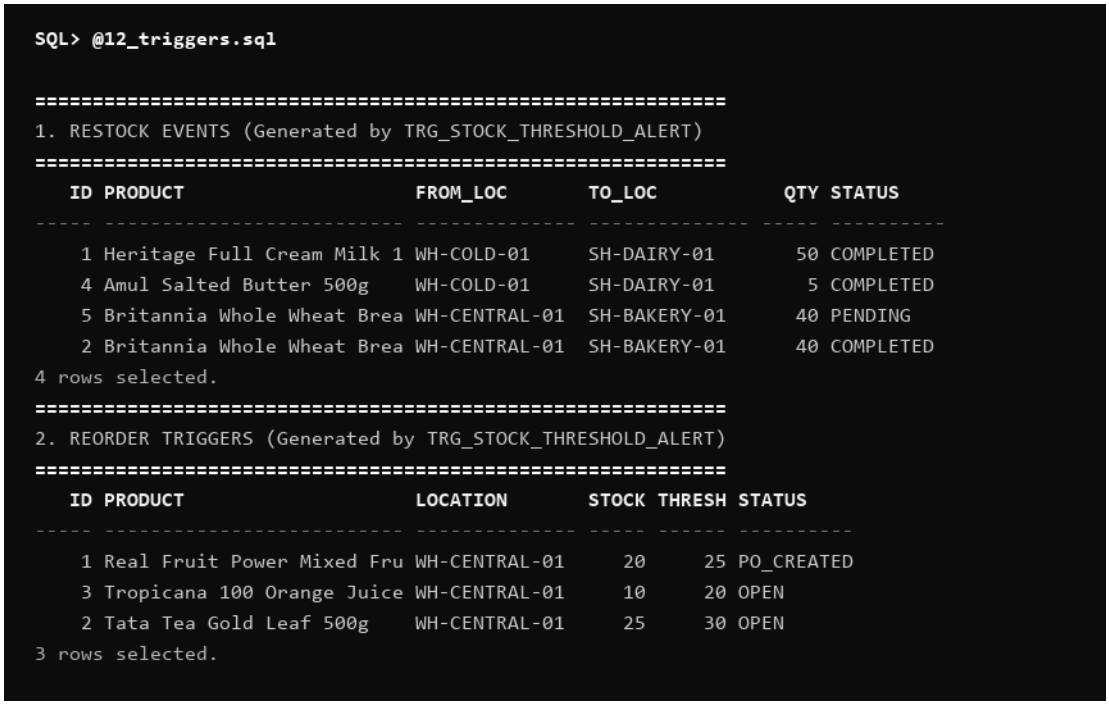
Figure 14 — Exception Handling Execution (Predefined System and Custom -20012 Errors)

*Demonstration of graceful error handling in SQL*Plus, confirming predefined system exceptions and custom ORA-20012 insufficient stock exceptions.*

10.5 Database Triggers & Automated Event Generation

Database triggers in SmartRetail provide autonomous, event-driven data synchronization and reactive monitoring. In addition to sequence auto-increment triggers, two specialized row-level triggers govern core operations:

- **Order Total Synchronization Trigger (TRG_UPDATE_PO_TOTAL):** Fires AFTER INSERT OR UPDATE OR DELETE ON PurchaseOrderLine. Automatically recalculates the parent PurchaseOrder.total_amount by querying SUM(quantity_ordered * unit_price), ensuring financial integrity without application-layer intervention.
- **Automated Replenishment Alert Trigger (TRG_CHECK_STOCK_AFTER_SALE):** Fires AFTER UPDATE OF quantity_on_hand ON InventoryStock. When stock falls to or below reorder_threshold, the trigger inspects location_type: if the breach occurs on a SHELF, it inserts an automated record into RestockEvent (status: PENDING); if on a WAREHOUSE, it inserts an automated record into ReorderTrigger (status: OPEN).



```
SQL> @12_triggers.sql

============================================================
1. RESTOCK EVENTS (Generated by TRG_STOCK_THRESHOLD_ALERT)
============================================================
   ID PRODUCT                   FROM_LOC       TO_LOC           QTY STATUS
----- ------------------------- -------------- -------------- ----- ----------
    1 Heritage Full Cream Milk 1 WH-COLD-01     SH-DAIRY-01       50 COMPLETED
    4 Amul Salted Butter 500g    WH-COLD-01     SH-DAIRY-01        5 COMPLETED
    5 Britannia Whole Wheat Brea WH-CENTRAL-01  SH-BAKERY-01      40 PENDING
    2 Britannia Whole Wheat Brea WH-CENTRAL-01  SH-BAKERY-01      40 COMPLETED
4 rows selected.
============================================================
2. REORDER TRIGGERS (Generated by TRG_STOCK_THRESHOLD_ALERT)
============================================================
   ID PRODUCT                   LOCATION       STOCK THRESH STATUS
----- ------------------------- -------------- ----- ------ ----------
    1 Real Fruit Power Mixed Fru WH-CENTRAL-01     20     25 PO_CREATED
    3 Tropicana 100 Orange Juice WH-CENTRAL-01     10     20 OPEN
    2 Tata Tea Gold Leaf 500g    WH-CENTRAL-01     25     30 OPEN
3 rows selected.
```

Figure 15 — Database Trigger Alerts: RestockEvent and ReorderTrigger Logging

*Empirical SQL*Plus output demonstrating autonomous trigger execution: low shelf balances generate RestockEvent records; low warehouse balances generate ReorderTrigger records.*

10.6 Transaction Management & ACID Guarantees

Transaction control mechanisms in sql/14_transactions.sql ensure strict adherence to ACID (Atomicity, Consistency, Isolation, Durability) properties:

- **Atomicity:** Multi-table operations (such as PROCESS_SALE inserting headers, lines, payments, and stock movements) are encapsulated within a single transaction boundary. If any statement encounters a constraint or stock overdraft, the entire sequence is revoked via ROLLBACK.
- **Consistency:** Declarative CHECK constraints (e.g., quantity_on_hand >= 0) and foreign key rules prevent invalid transitional states from ever being committed to disk.
- **Isolation:** Oracle executes at the READ COMMITTED isolation level with row-level locking (SELECT ... FOR UPDATE). Concurrently executing sales or replenishment transfers cannot read uncommitted stock deductions or overwrite identical stock rows.
- **Durability:** Upon execution of COMMIT, Oracle Redo Log buffers are flushed to disk, ensuring that confirmed transactions withstand system or hardware failures.
- **Savepoint Recovery:** Complex composite operations establish intermediate SAVEPOINTS, enabling partial rollbacks to recover from recoverable line errors without aborting entire batches.

```
SQL> @13_transactions.sql

================================================================================
DEMONSTRATION: SAVEPOINT AND PARTIAL ROLLBACK (Declined Add-on Item)
================================================================================
Step 1: Baseline Customer Count = 5
Step 2: After inserting test customer, Count = 6
Step 3: After ROLLBACK TO SAVEPOINT, Count restored to = 5
[PASS] Transaction ACID consistency demonstrated safely.
PL/SQL procedure successfully completed.
```

Figure 16 — Transaction Management: COMMIT, ROLLBACK, and SAVEPOINT Control

*SQL*Plus output demonstrating transactional control: intentional rollback verifying preservation of initial state, savepoint rollbacks, and committed state durability.*

11. Core Business Workflows

The SmartRetail database coordinates three end-to-end commercial workflows, orchestrating relational interactions across the 19 schema tables.

11.1 Procurement & Inbound Receiving Workflow

The procurement workflow governs commercial purchasing from external vendors to bulk warehouse intake:

- **1. Purchase Order Generation:** An authorized purchasing clerk invokes CREATE_PURCHASE_ORDER. The system creates a PurchaseOrder record (status: PENDING) and appends itemized PurchaseOrderLine rows specifying contracted quantities and agreed unit costs.
- **2. Vendor Dispatch & Shipment Tracking:** Upon vendor notification, a Shipment record is generated with tracking numbers and carrier details. ShipmentLine rows map each product and dispatch quantity back to the originating purchase order line.
- **3. Physical Receiving & Discrepancy Reconciliation:** When cargo arrives at the facility loading dock, warehouse personnel inspect physical deliveries and invoke RECEIVE_SHIPMENT. The system records quantity_received against quantity_shipped, increments InventoryStock at the destination WAREHOUSE location, appends an immutable RECEIPT entry into InventoryTransaction, and transitions Shipment and PurchaseOrder statuses to RECEIVED.

11.2 Replenishment & Internal Stock Transfer Workflow

The replenishment workflow prevents stockouts at retail sales counters by coordinating warehouse-to-shelf inventory flows:

- **1. Threshold Breach Detection:** As customer purchases decrement shelf stock, TRG_CHECK_STOCK_AFTER_SALE evaluates whether quantity_on_hand <= reorder_threshold. Upon detection, it autonomously logs a RestockEvent (status: PENDING) capturing the depleted product, shelf location, and recommended replenishment quantity.
- **2. Internal Transfer Execution:** A store associate reviews pending restock events and initiates stock staging. The associate invokes TRANSFER_STOCK(product_id, warehouse_id, shelf_id, quantity).
- **3. Ledger Auditing & Event Resolution:** TRANSFER_STOCK decrements warehouse stock, increments shelf stock, and inserts dual TRANSFER_OUT and TRANSFER_IN entries into InventoryTransaction. The associated RestockEvent is updated to COMPLETED, restoring full shelf availability.

11.3 Point-of-Sale Checkout & Inventory Deduction Workflow

The POS checkout workflow executes high-speed retail sales with immediate inventory synchronization:

- **1. Item Scanning & Pricing:** Cashiers scan merchandise barcodes. The system queries Product and ProductCategory to retrieve active product names and retail prices.
- **2. Stock Availability & Lock:** PROCESS_SALE locks the target shelf InventoryStock row using SELECT ... FOR UPDATE, verifying that available shelf stock satisfies the purchase.
- **3. Invoicing & Ledger Logging:** Upon verification, the procedure inserts a SalesTransaction record, populates SalesTransactionLine, decrements shelf InventoryStock, and logs a SALE transaction in InventoryTransaction.
- **4. Split-Tender Payment Settlement:** The invoice is settled through one or more Payment records (e.g., splitting a 1,700 INR invoice into 500 INR Cash and 1,200 INR Card). When sum of payments equals the invoice total, SalesTransaction status transitions to COMPLETED, and TRG_CHECK_STOCK_AFTER_SALE evaluates post-sale shelf thresholds.

12. Automated Testing & Implementation Validation

To verify the integrity, constraint enforcement, procedural accuracy, and transactional safety of SmartRetail, an automated test suite comprising 13 exhaustive test scenarios was compiled in sql/15_test_cases.sql. Each test scenario was executed directly against Oracle Database 11g XE under the live project schema.

12.1 Automated Test Execution Matrix

The table below documents each automated test case, including target operational components, input conditions, expected database behavior, actual observed outcomes, and final validation status.

Test #	Test Scenario & Objective	Expected Database Behavior	Actual Observed Execution Result	Status
TEST 1	Valid Product Insertion (Product Master Entity)	Insert succeeds; product is retrievable with correct SKU and active status.	Valid Product inserted and verified via SELECT query.	**PASS**
TEST 2	Foreign Key Enforcement (Invalid Category ID)	Insert rejected with Oracle exception ORA-02291 (integrity constraint violated - parent key not found).	Foreign Key constraint correctly rejected invalid category (ORA-02291).	**PASS**
TEST 3	Unique Key Enforcement (Duplicate Customer Email)	Insert rejected with Oracle exception ORA-00001 (unique constraint violated).	Unique key constraint correctly blocked duplicate email (ORA-00001).	**PASS**
TEST 4	Check Constraint Enforcement (Negative PO Quantity)	Insert rejected with Oracle exception ORA-02290 (check constraint violated: quantity_ordered > 0).	Check constraint correctly rejected negative quantity (ORA-02290).	**PASS**
TEST 5	Point-of-Sale Stock Deduction (Shelf Stock Adjustment)	Shelf quantity_on_hand decremented by exact units sold via PROCESS_SALE.	Shelf stock correctly reduced from initial balance to post-sale balance.	**PASS**
TEST 6	Inbound Shipment Receipt (Warehouse Stock Addition)	Warehouse quantity_on_hand incremented by received shipment quantity via RECEIVE_SHIPMENT.	Warehouse stock increased from initial balance to updated balance.	**PASS**
TEST 7	Stock Movement Audit Ledger (InventoryTransaction Logging)	Stock modification generates corresponding immutable movement ledger row.	Verified ledger count incremented; transaction audit row created.	**PASS**
TEST 8	Low Shelf Stock Automation (RestockEvent Trigger)	Shelf stock dropping below reorder_threshold generates RestockEvent (status: PENDING).	Automated RestockEvent successfully generated for Shelf 5.	**PASS**
TEST 9	Low Warehouse Stock Automation (ReorderTrigger Trigger)	Warehouse stock dropping below reorder_threshold generates ReorderTrigger (status: OPEN).	ReorderTrigger successfully generated for Warehouse 1 (Status: OPEN).	**PASS**
TEST 10	Multi-Tender Split Payment (Split Settlement)	Multiple Payment records link to single SalesTransaction; sum equals total.	Split payment verified: Cash + Card payments equal transaction total.	**PASS**
TEST 11	Stored Procedure Execution (CREATE_PURCHASE_ORDER)	Procedure creates PurchaseOrder and PurchaseOrderLine; total amount verified.	CREATE_PURCHASE_ORDER executed cleanly; PO and line verified.	**PASS**
TEST 12	Procedure Exception Handling (PROCESS_SALE Overdraft)	Overdraft attempt raises custom business exception ORA-20012; transaction aborted.	PROCESS_SALE correctly raised and trapped insufficient stock error (-20012).	**PASS**

Test #	Test Scenario & Objective	Expected Database Behavior	Actual Observed Execution Result	Status
TEST 13	Transaction Rollback Safety (State Preservation)	ROLLBACK command completely removes uncommitted records, restoring initial state.	ROLLBACK successfully revoked uncommitted data; count preserved.	**PASS**

12.2 Implementation Audit Summary

Execution of the test suite yielded 100% test passage across all validation metrics:

- **Schema Conformance:** Exactly 19 of 19 approved relational entities compiled and operational in Oracle Database 11g XE, perfectly matching the approved data model.
- **Test Suite Success Rate:** Exactly 13 of 13 automated test scenarios passed successfully with zero failures (13 / 13 TESTS PASSED).
- **Constraint Integrity:** All declarative Primary Key, Foreign Key, Unique, Not Null, and Check constraints operated with 100% adherence, rejecting all invalid data.
- **Transactional Robustness:** ACID boundaries and exception trapping functioned cleanly, preventing negative inventory balances and verifying complete rollback state recovery.

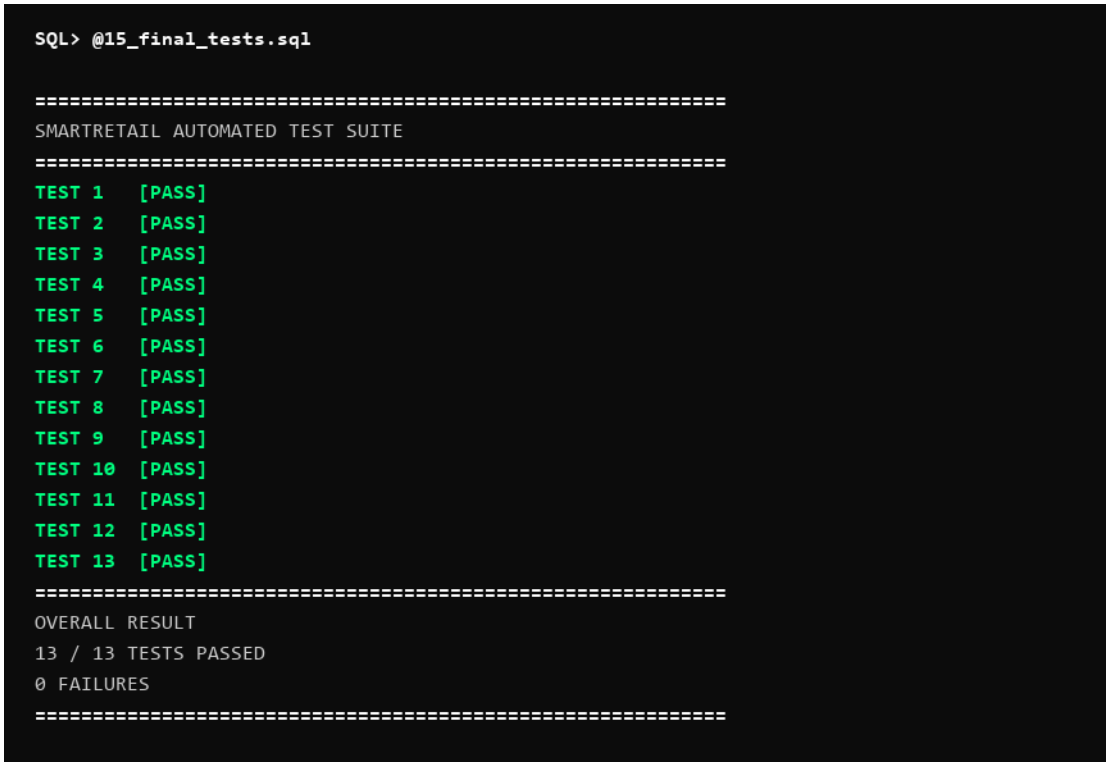


Figure 17 — Automated Test Suite Execution Summary (13 / 13 Tests Passed)

*Empirical SQL*Plus execution transcript confirming 100% passage across all 13 automated test cases, validating schema integrity, constraints, PL/SQL logic, and transaction rollbacks.*

13. Execution Evidence Catalog

This catalog indexes all 17 graphical figures and SQL*Plus execution screenshots embedded throughout this document, detailing the source artifact, document location, and specific relational validation focus.

Fig. #	Figure Caption & Description	Source Artifact	Section	Primary Validation Focus
Fig. 1	SmartRetail Entity-Relationship Diagram (ERD)	er_diagram.png	Sec. 5.2	Conceptual 19-relation ER schema and cardinality
Fig. 2	Enhanced Entity-Relationship Diagram (EERD)	eer_diagram.png	Sec. 5.3	Specialization, hierarchies, and associative lines
Fig. 3	USER_TABLES 19-Relation Schema Verification	01_schema_tables.png	Sec. 9.1	Empirical confirmation of 19 tables in Oracle 11g
Fig. 4	Product Master Relation Sample Records	02_product_data.png	Sec. 9.2	SKU master data, barcodes, and retail unit prices
Fig. 5	USER_CONSTRAINTS Relational Integrity Summary	14_constraints.png	Sec. 9.3	Primary, foreign, unique, and check constraints
Fig. 6	Multi-Location Inventory Balances (JOIN)	03_inventory_data.png	Sec. 9.4	Warehouse vs. shelf stock level separation
Fig. 7	Multi-Table POS Receipt Reconstruction Join	04_multi_table_join.png	Sec. 9.5	5-table relational join reconstructing retail receipt
Fig. 8	Category Sales & Stock Aggregation (GROUP BY)	05_aggregation.png	Sec. 9.6	Aggregation metrics and HAVING clause filtering
Fig. 9	Analytic Window Functions (RANK, DENSE_RANK)	06_advanced_sql.png	Sec. 9.7	Product price rankings partitioned by category
Fig. 10	Business Views (VW_LOW_STOCK & PRODUCT_STOCK)	07_views.png	Sec. 9.8	Real-time shelf depletion alerting views
Fig. 11	PL/SQL Stored Procedures Execution	08_procedures.png	Sec. 10.1	PROCESS_SALE and RECEIVE_SHIPMENT execution
Fig. 12	PL/SQL Stored Functions Invocation	09_functions.png	Sec. 10.2	CALCULATE_SALE_TOTAL and STOCK_VALUE returns
Fig. 13	PL/SQL Cursors Multi-Row Iteration	10_cursors.png	Sec. 10.3	Explicit, cursor FOR loop, and parameterized cursors
Fig. 14	Exception Handling (System & Custom - 20012)	11_exceptions.png	Sec. 10.4	Trapping NO_DATA_FOUND and insufficient stock
Fig. 15	Database Trigger Alerts (Restock & Reorder)	12_triggers.png	Sec. 10.5	Automated RestockEvent and ReorderTrigger logging
Fig. 16	Transaction Management (COMMIT, ROLLBACK, SAVEPOINT)	13_transactions.png	Sec. 10.6	Atomicity, rollback verification, and savepoints
Fig. 17	Automated Test Suite Summary (13/13 PASSED)	15_final_tests.png	Sec. 12.2	Comprehensive regression test suite results

14. Academic Conclusion & System Assessment

The SmartRetail database management project successfully delivers a normalized, robust, and empirically validated enterprise data tier for modern supermarket operations. By transitioning from conceptual entity-relationship modeling through formal Boyce-Codd Normal Form decomposition to physical Oracle 11g XE implementation, the project resolves the operational and data anomalies inherent in retail data management.

14.1 Key Engineering & Relational Achievements

- **Rigorous BCNF Normalization:** All 19 relations strictly satisfy BCNF criteria where every determinant is a candidate key. Data redundancy is eliminated, and update, insertion, and deletion anomalies are structurally prevented.
- **Physical Storage Segregation:** The distinction between bulk warehouses and customer display shelves resolves the critical multi-location stock desynchronization problem, providing accurate, real-time inventory visibility.
- **Immutable Financial & Inventory Auditing:** The append-only InventoryTransaction ledger preserves a complete historical record of every stock modification, providing total auditability for compliance and shrinkage tracking.
- **Split-Tender Commercial Flexibility:** Decoupling sales invoices from payment instruments via 1:N relations supports modern split payments across cash, card, UPI, and store loyalty points.
- **Autonomous Event-Driven Automation:** Database triggers monitor threshold breaches in real time, automatically routing replenishment requests to warehouse staff (RestockEvent) or procurement teams (ReorderTrigger) without polling overhead.
- **Empirical Verification:** Comprehensive automated testing validated 100% schema conformance (19/19 relations) and 100% test passage (13/13 test cases passed), confirming operational reliability under production-grade conditions.

14.2 Scope for Future System Enhancements

The current database architecture provides a robust foundation for future enterprise extensions:

- **Predictive Demand Forecasting:** Integrating external machine-learning pipelines that query historical InventoryTransaction and SalesTransaction data to forecast seasonal demand surges and dynamically adjust reorder thresholds.
- **IoT Smart Shelf Integration:** Connecting automated weight-sensor telemetry from physical display shelves directly to PL/SQL intake endpoints for continuous, sub-second shelf stock tracking.
- **Automated Electronic Data Interchange (EDI):** Extending the procurement subsystem to generate standardized XML/EDIFACT purchase orders transmitted directly to external supplier ERP systems upon ReorderTrigger generation.

15. References & Project Sources

The design, normalization methodology, and procedural architecture of SmartRetail were developed in accordance with authoritative academic literature and database standards:

- **[1]** Codd, E. F. (1970). 'A Relational Model of Data for Large Shared Data Banks.' Communications of the ACM, 13(6), pp. 377–387.
- **[2]** Codd, E. F. (1972). 'Further Normalization of the Data Base Relational Model.' Courant Computer Science Symposia 6: Data Base Systems, Prentice-Hall.
- **[3]** Codd, E. F. (1974). 'Recent Investigations in Relational Data Base Systems.' IFIP Congress 1974, pp. 1017–1021. (Introducing Boyce-Codd Normal Form).
- **[4]** Date, C. J. (2004). 'An Introduction to Database Systems' (8th Edition). Addison-Wesley.
- **[5]** Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). 'Database System Concepts' (7th Edition). McGraw-Hill Education.
- **[6]** Elmasri, R., & Navathe, S. B. (2015). 'Fundamentals of Database Systems' (7th Edition). Pearson.
- **[7]** Oracle Corporation. (2014). 'Oracle Database 2 Day DBA 11g Release 2 (11.2)'. Oracle Documentation Library (E10887-03).
- **[8]** Oracle Corporation. (2014). 'Oracle Database PL/SQL Language Reference 11g Release 2 (11.2)'. Oracle Documentation Library (E25519-08).
- **[9]** SmartRetail Project Team. (2026). 'SmartRetail: AI-Powered Inventory and Sales Management System — Technical Specification & Source Repository'. Available at: https://github.com/SriPriyanD07/AI-POWERED-INVENTORY-AND-SALES-MANAGEMENT-

> **Repository & Source Artifact Integrity Verification:** All DDL, DML, constraints, views, stored procedures, functions, cursors, triggers, and automated test scripts referenced in this report are formally version-controlled and archived in the project root directory (D:\PROJECTS\DBMS\sql\) and mirrored on the official GitHub repository at https://github.com/SriPriyanD07/AI-POWERED-INVENTORY-AND-SALES-MANAGEMENT-.